

Homework 7 : GANs

In this assignment, you will learn to write an advanced PyTorch implementation concept commonly used in a complex deep learning pipeline by using GAN as a learning example.

You will also start working with more complex architectures (upsampling) and writing style (complex modules such as modulelist) for pytorch.

Every TODO is weighted equally. Optional TODO is half of a regular TODO.

Part 1 : WGAN-GP reimplementation

In this section, you are going to reimplement WGAN-GP (<https://arxiv.org/pdf/1704.00028.pdf>) based on the pseudocode provided in the paper to generate MNIST digit characters. Some parts are intentionally modified to discourage straight copy-pasting from public repositories.

Algorithm 1 WGAN with gradient penalty. We use default values of $\lambda = 10$, $n_{\text{critic}} = 5$, $\alpha = 0.0001$, $\beta_1 = 0$, $\beta_2 = 0.9$.

Require: The gradient penalty coefficient λ , the number of critic iterations per generator iteration n_{critic} , the batch size m , Adam hyperparameters α, β_1, β_2 .

Require: initial critic parameters w_0 , initial generator parameters θ_0 .

```

1: while  $\theta$  has not converged do
2:   for  $t = 1, \dots, n_{\text{critic}}$  do
3:     for  $i = 1, \dots, m$  do
4:       Sample real data  $\mathbf{x} \sim \mathbb{P}_r$ , latent variable  $\mathbf{z} \sim p(\mathbf{z})$ , a random number  $\epsilon \sim U[0, 1]$ .
5:        $\tilde{\mathbf{x}} \leftarrow G_{\theta}(\mathbf{z})$ 
6:        $\hat{\mathbf{x}} \leftarrow \epsilon \mathbf{x} + (1 - \epsilon) \tilde{\mathbf{x}}$ 
7:        $L^{(i)} \leftarrow D_w(\tilde{\mathbf{x}}) - D_w(\mathbf{x}) + \lambda(\|\nabla_{\hat{\mathbf{x}}} D_w(\hat{\mathbf{x}})\|_2 - 1)^2$ 
8:     end for
9:      $w \leftarrow \text{Adam}(\nabla_w \frac{1}{m} \sum_{i=1}^m L^{(i)}, w, \alpha, \beta_1, \beta_2)$ 
10:   end for
11:   Sample a batch of latent variables  $\{\mathbf{z}^{(i)}\}_{i=1}^m \sim p(\mathbf{z})$ .
12:    $\theta \leftarrow \text{Adam}(\nabla_{\theta} \frac{1}{m} \sum_{i=1}^m -D_w(G_{\theta}(\mathbf{z})), \theta, \alpha, \beta_1, \beta_2)$ 
13: end while

```

The pseudocode could be organized into two main parts: discriminator optimization in line 2-10, and generator optimization in line 11-12.

The discriminator part consists of four steps:

- Line 4: data, and noise sampling with a batch size of m
- Line 5-7: discriminator loss calculation
- Line 9: discriminator update
- Repeat line 4-9 for n_{critic} steps

After the discriminator is updated, the generator is then updated by performing two steps:

- Line 11: noise sampling
- Line 12: generator loss calculation and update

This part is divided into four subsections: network initialization, hyperparameter initialization, data preparation, and training loop. The detail for each part will be explained in the subsections.

Downloading MNIST dataset

The MNIST dataset contains 60,000 training digit character image (0-9) at 28x28 resolution that are normalized to [0, 1]. Given the training images, your task is to generate new training images using WGAN-GP by learning from the training distribution.

```
In [17]: import torchvision.datasets as datasets
import numpy as np

mnist_trainset = datasets.MNIST(root='./data', train=True, download=True, transform=None)
trainX = np.array(mnist_trainset.data[... , None]).transpose(0, 3, 1, 2) / 255
print("Dataset size : ", trainX.shape)
```

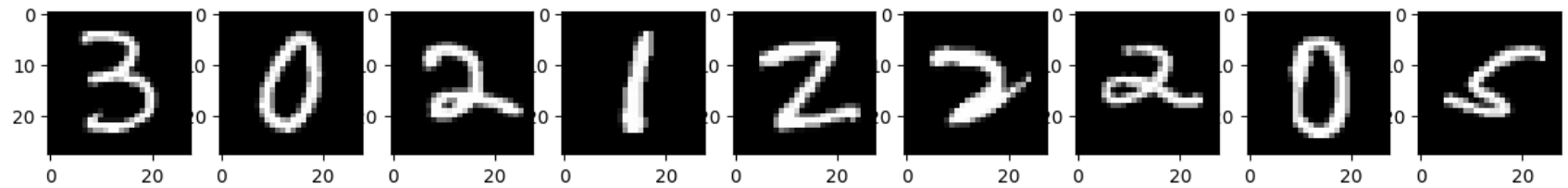
Dataset size : (60000, 1, 28, 28)

/tmp/ipykernel_18127/660608400.py:5: DeprecationWarning: __array__ implementation doesn't accept a copy keyword, so passing copy=False failed. __array__ must implement 'dtype' and 'copy' keyword arguments. To learn more, see the migration guide https://numpy.org/devdocs/numpy_2_0_migration_guide.html#adapting-to-changes-in-the-copy-keyword

```
trainX = np.array(mnist_trainset.data[... , None]).transpose(0, 3, 1, 2) / 255
```

Dataset Visualization

```
In [18]: import matplotlib.pyplot as plt
import numpy as np
plt.figure(figsize = (15,75))
for i in range(9):
    plt.subplot( int('19{}'.format(i+1)) )
    plt.imshow( trainX[np.random.randint(len(trainX))].transpose((1, 2, 0))[... , 0] , cmap = 'gray' )
plt.show()
```



Generator and Discriminator network

Before training, the deep learning networks have to be initialized first. Therefore, in this part, you are going to write a generator and discriminator network based on the description provided below.

The description of the discriminator network is shown in the Table below.

Discriminator ($D(x)$)			
	Kernel size	Resample	Output shape
ConvBlock	5×5	Down	$128 \times 14 \times 14$
ConvBlock	5×5	Down	$256 \times 7 \times 7$
ConvBlock	5×5	Down	$512 \times 4 \times 4$
Linear	-	-	1

The network also has some specific requirements:

- ConvBlock is a Convolution-ReLU layer
- All ReLUs in the encoder are leaky, with a slope of 0.1

The description of the generator network is shown in the Table below.

Generator (G(z))			
	Kernel size	Resample	Output shape
z	-	-	128
Linear	-	-	$512 \times 4 \times 4$
ConvBlock	5×5	Up	$256 \times 8 \times 8$
ConvBlock	5×5	Up	$128 \times 16 \times 16$
Conv, Sigmoid	5×5	Up	$1 \times 32 \times 32$
-	-	Down	$1 \times 28 \times 28$

The network also has some specific requirements:

- ConvBlock is a ConvTranspose-BatchNorm-ReLU layer
- Downsampling method is bilinear interpolation (torch.nn.Upsample or torch.nn.functional.interpolate)

TODO 1: Implement a discriminator network.

TODO 2: Implement a generator network.

```
In [10]: import torch
import torch.nn.functional as F
from torch import nn
from torchvision import transforms

class Discriminator(nn.Module):
    ##TODO1 implement the discriminator (critic)
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(1, 128, 5, 2, 2)
        self.conv2 = nn.Conv2d(128, 256, 5, 2, 2)
        self.conv3 = nn.Conv2d(256, 512, 5, 2, 2)
        self.fc = nn.Linear(512*4*4, 1)
```

```

def forward(self, x):
    hd1 = F.leaky_relu(self.conv1(x), 0.1)
    hd2 = F.leaky_relu(self.conv2(hd1), 0.1)
    hd3 = F.leaky_relu(self.conv3(hd2), 0.1)
    out = self.fc(hd3.flatten(1)).reshape(-1, 1, 1, 1)
    return out

class GeneratorBlock(nn.Module):
    def __init__(self, in_channels, out_channels, out_size):
        super().__init__()
        # 1. Bilinear upsampling to the target size
        self.upsample = nn.Upsample(size=out_size, mode='bilinear', align_corners=False)
        # 2. Convolution (kernel=5, padding=2), BatchNorm, ReLU
        self.conv = nn.Sequential(
            nn.Conv2d(in_channels, out_channels, kernel_size=5, stride=1, padding=2),
            nn.BatchNorm2d(out_channels),
            nn.ReLU(inplace=True)
        )

    def forward(self, x):
        x = self.upsample(x)
        return self.conv(x)

class Generator(nn.Module):
    def __init__(self, z_dim=128):
        super().__init__()

        # 1-2. Linear layer mapping from z(128) to 512x4x4
        self.fc = nn.Sequential(
            nn.Linear(z_dim, 512 * 4 * 4),
            nn.BatchNorm1d(512 * 4 * 4),
            nn.ReLU(inplace=True)
        )

        # 3. ConvBlock Resample Up
        self.block1 = GeneratorBlock(512, 256, out_size=(8, 8))

        # 4. ConvBlock Resample Up
        self.block2 = GeneratorBlock(256, 128, out_size=(16, 16))

        # 5. Conv, Sigmoid Resample Up to 32x32
        self.upsample3 = nn.Upsample(size=(32, 32), mode='bilinear', align_corners=False)
        self.conv_out = nn.Sequential(

```

```

        nn.Conv2d(128, 1, kernel_size=5, stride=1, padding=2),
        nn.Sigmoid() # Notice it asks for Sigmoid here! (outputs [0, 1])
    )

    # 6. Final Resample Down to 28x28 (Matching MNIST dimension)
    self.final_downsample = nn.Upsample(size=(28, 28), mode='bilinear', align_corners=False)

    def forward(self, z):
        # FC and reshape
        x = self.fc(z)
        x = x.view(x.size(0), 512, 4, 4)

        # Main upsampling blocks
        x = self.block1(x)
        x = self.block2(x)

        # Final conv + sigmoid block (to 32x32)
        x = self.upsample3(x)
        x = self.conv_out(x)

        # Final crop/downsample back to 28x28
        x = self.final_downsample(x)
        return x

```

```

In [11]: discriminator = Discriminator().cuda()
         generator = Generator().cuda()

```

Network verification

TODO 3: What is the input and output shape of the generator and discriminator network? Verify that the implemented networks are the same as the answer you have provided.

```

In [8]: from torchinfo import summary

```

```

In [12]: print(summary(discriminator, input_size=(6000,1,28,28)))
         print(summary(generator, input_size=(6000, 128)))

```

Layer (type:depth-idx)	Output Shape	Param #
Discriminator	[6000, 1, 1, 1]	--
└─Conv2d: 1-1	[6000, 128, 14, 14]	3,328
└─Conv2d: 1-2	[6000, 256, 7, 7]	819,456
└─Conv2d: 1-3	[6000, 512, 4, 4]	3,277,312
└─Linear: 1-4	[6000, 1]	8,193
Total params: 4,108,289		
Trainable params: 4,108,289		
Non-trainable params: 0		
Total mult-adds (Units.GIGABYTES): 559.50		
Input size (MB): 18.82		
Forward/backward pass size (MB): 2199.60		
Params size (MB): 16.43		
Estimated Total Size (MB): 2234.85		
Layer (type:depth-idx)	Output Shape	Param #
Generator	[6000, 1, 28, 28]	--
└─Sequential: 1-1	[6000, 8192]	--
└─Linear: 2-1	[6000, 8192]	1,056,768
└─BatchNorm1d: 2-2	[6000, 8192]	16,384
└─ReLU: 2-3	[6000, 8192]	--
└─GeneratorBlock: 1-2	[6000, 256, 8, 8]	--
└─Upsample: 2-4	[6000, 512, 8, 8]	--
└─Sequential: 2-5	[6000, 256, 8, 8]	--
└─Conv2d: 3-1	[6000, 256, 8, 8]	3,277,056
└─BatchNorm2d: 3-2	[6000, 256, 8, 8]	512
└─ReLU: 3-3	[6000, 256, 8, 8]	--
└─GeneratorBlock: 1-3	[6000, 128, 16, 16]	--
└─Upsample: 2-6	[6000, 256, 16, 16]	--
└─Sequential: 2-7	[6000, 128, 16, 16]	--
└─Conv2d: 3-4	[6000, 128, 16, 16]	819,328
└─BatchNorm2d: 3-5	[6000, 128, 16, 16]	256
└─ReLU: 3-6	[6000, 128, 16, 16]	--
└─Upsample: 1-4	[6000, 128, 32, 32]	--
└─Sequential: 1-5	[6000, 1, 32, 32]	--
└─Conv2d: 2-8	[6000, 1, 32, 32]	3,201


```

|      └Sigmoid: 2-9          [6000, 1, 32, 32]      --
|└Upsample: 1-6              [6000, 1, 28, 28]      --

```

```

=====
Total params: 5,173,505
Trainable params: 5,173,505
Non-trainable params: 0
Total mult-adds (Units.TERABYTES): 2.54
=====

```

```

=====
Input size (MB): 3.07
Forward/backward pass size (MB): 5554.18
Params size (MB): 20.69
Estimated Total Size (MB): 5577.94
=====

```

Parameter Initialization

After the network is initialized, we then set up training hyperparameters for the training. In this part, hyperparameters have already been partially provided in the cell below, though some of them are intentionally left missing (**None**). Your task is to fill the missing parameters based on the pseudocode above.

TODO4: Initialize the missing model hyperparameters and optimizers based on the pseudocode above.

Note: To hasten the training process of our toy experiment, the training step and batch size is reduced to 3000 and 32, respectively.

```
In [14]: NUM_ITERATION = 3000
BATCH_SIZE = 32
fixed_z = torch.randn((8, 128)).cuda()
def schedule(i):
    lr = 1e-4
    if(i > 2500): lr *= 0.1
    return lr
losses = {'D' : [None], 'G' : [None]}

## TODO4 initialize missing hyperparameter and optimizer
G_optimizer = torch.optim.Adam(generator.parameters(), lr=1e-4, betas=(0.0, 0.9))
D_optimizer = torch.optim.Adam(discriminator.parameters(), lr=1e-4, betas=(0.0, 0.9))
GP_lambda = 10
n_critic = 5
```

Data preparation

TODO 5: Create a dataloader that could generate the data in line 4. The dataloader should return $\mathbf{x}, \mathbf{z}, \epsilon$ with a batch size of `BATCH_SIZE`

```
In [15]: from torch.utils.data import DataLoader, Dataset
```

```
In [20]: # TODO5 implement dataloader
class MNISTDataset(Dataset):
    def __init__(self, data, z_dim=128):
        self.data = torch.tensor(data, dtype=torch.float32)
        self.z_dim = z_dim
    def __len__(self):
        return len(self.data)

    def __getitem__(self, idx):
        x = self.data[idx]
        z = torch.randn(self.z_dim)
        epsilon = torch.rand(1, 1, 1)
        return x, z, epsilon

mnist_dataset = MNISTDataset(trainX, z_dim=128)
train_loader = DataLoader(mnist_dataset, batch_size=BATCH_SIZE, shuffle=True, pin_memory=True)
```

Training loop

This section is the place where the training section starts. **It is highly recommended that you understand the pseudocode before performing the tasks below.** To train the WGAN-GP you have to perform the following tasks:

TODO6: Update the learning rate base on the provided scheduler.

TODO7: Sample the data from the dataloader (Line 4).

TODO8 : Calcualte the discriminator loss (Line 5-7).

- In the line 7 you have to implement the gradient penalty term $\lambda(\|\nabla_{\hat{\mathbf{x}}} D_w(\hat{\mathbf{x}})\|_2 - 1)^2$, which is a custom gradient. You may read <https://pytorch.org/docs/stable/generated/torch.autograd.grad.html> to find how custom gradient is implemented.
- HINT: Gradient norm calculation is still part of the computation graph.

TODO9: Update the discriminator loss (Line 9).

TODO10: Calculate and update the generator loss (Line 11-12).

If your implementation is correct, the generated images should resemble an actual digit character after 500 iterations.

```
In [21]: from tqdm import tqdm

for i in tqdm(range(NUM_ITERATION)):
    ## TODO6 update learning rate
    D_optimizer.param_groups[0]['lr'] = schedule(i)
    G_optimizer.param_groups[0]['lr'] = schedule(i)
    data_iterator = iter(train_loader)

    for t in range(n_critic):
        ## TODO7 line 4: sample data from dataloader
        try:
            x, z, e = next(data_iterator)
        except StopIteration:
            data_iterator = iter(train_loader)
            x, z, e = next(data_iterator)

        real = x.cuda()
        z = z.cuda()
        epsilon = e.cuda()
```

```

with torch.no_grad():
    fake = generator(z)

## TODO8 line5-7 : calculate discriminator loss
critic_real = discriminator(real)
critic_fake = discriminator(fake)

# Calculate Gradient Penalty
mixed_images = (real * epsilon + fake * (1 - epsilon)).requires_grad_(True)
mixed_scores = discriminator(mixed_images)
gradient = torch.autograd.grad(
    inputs=mixed_images,
    outputs=mixed_scores,
    grad_outputs=torch.ones_like(mixed_scores),
    create_graph=True,
    retain_graph=True
)[0]
gradient = gradient.view(gradient.shape[0], -1)
gradient_norm = gradient.norm(2, dim=1)
gp = torch.mean((gradient_norm - 1) ** 2)

loss_critic = torch.mean(critic_fake) - torch.mean(critic_real) + GP_lambda * gp

## TODO9 : line 9 update discriminator loss
D_optimizer.zero_grad()
loss_critic.backward()
D_optimizer.step()

## TODO10 : line 11-12 calculate and update the generator loss
try:
    _, z_gen, _ = next(data_iterator)
except StopIteration:
    data_iterator = iter(dataloader)
    _, z_gen, _ = next(data_iterator)

z_gen = z_gen.cuda()
gen_fake = generator(z_gen)

# For WGAN, you want to MAXIMIZE critic's predicted validity of fake images.
loss_gen = -torch.mean(discriminator(gen_fake))

G_optimizer.zero_grad()

```

```

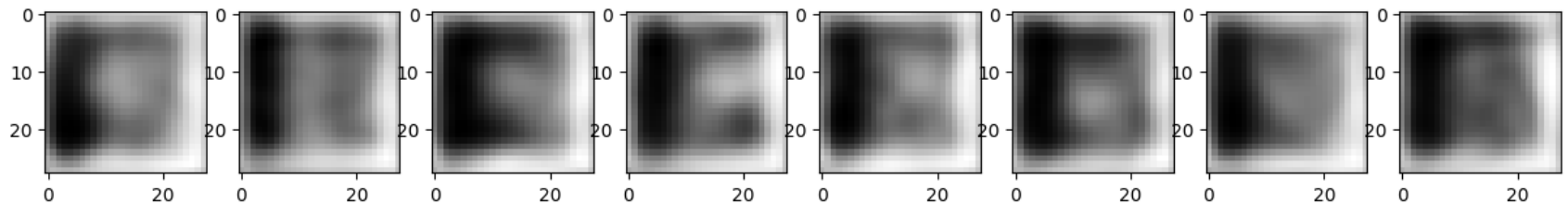
loss_gen.backward()
G_optimizer.step()

losses['D'].append(loss_critic.item())
losses['G'].append(loss_gen.item())

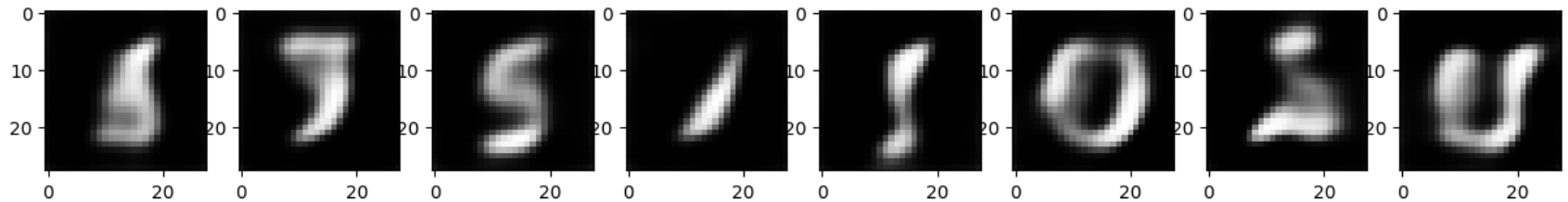
# Output visualization : If your reimplementation is correct, the generated images should start resembling
if(i % 100 == 0):
    plt.figure(figsize = (15,75))
    print(f"Discriminator Loss: {losses['D'][-1]:.4f} | Generator Loss: {losses['G'][-1]:.4f}")
    with torch.no_grad():
        res = generator(fixed_z).cpu().detach().numpy()
        for k in range(8):
            plt.subplot( int('18{}'.format(k+1)) )
            plt.imshow( res[k].transpose(1, 2, 0)[..., 0], cmap = 'gray' )
    plt.show()

```

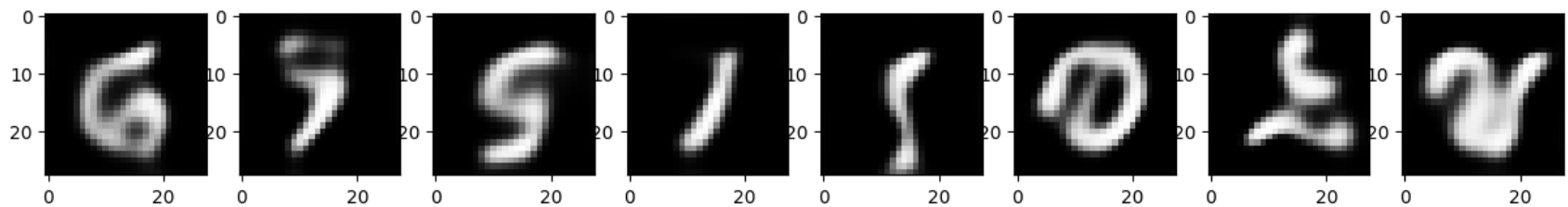
0%| | 0/3000 [00:00<?, ?it/s]
Discriminator Loss: 5.7492 | Generator Loss: 1.8890



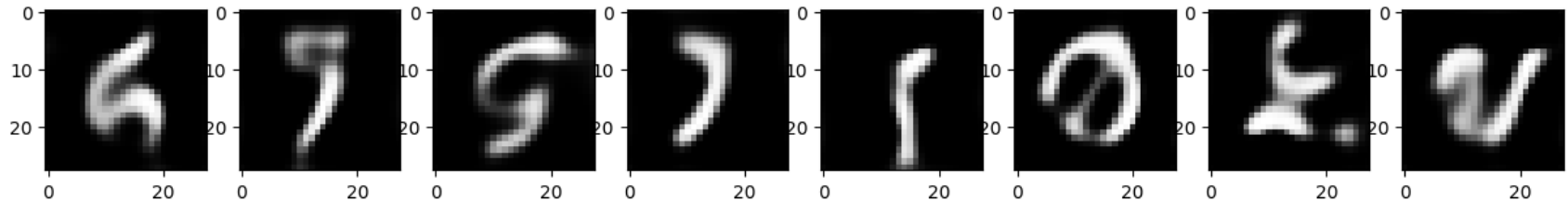
3%| | 99/3000 [00:07<03:27, 13.99it/s]
Discriminator Loss: -4.0903 | Generator Loss: -1.9862



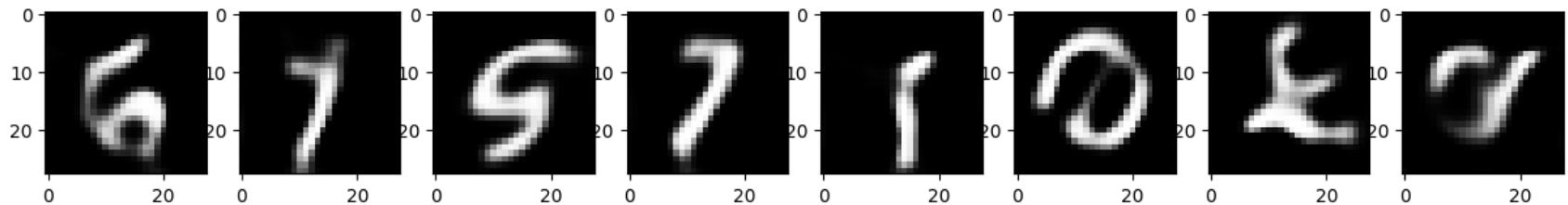
7%| | 199/3000 [00:15<03:21, 13.87it/s]
Discriminator Loss: -3.0987 | Generator Loss: -2.0627



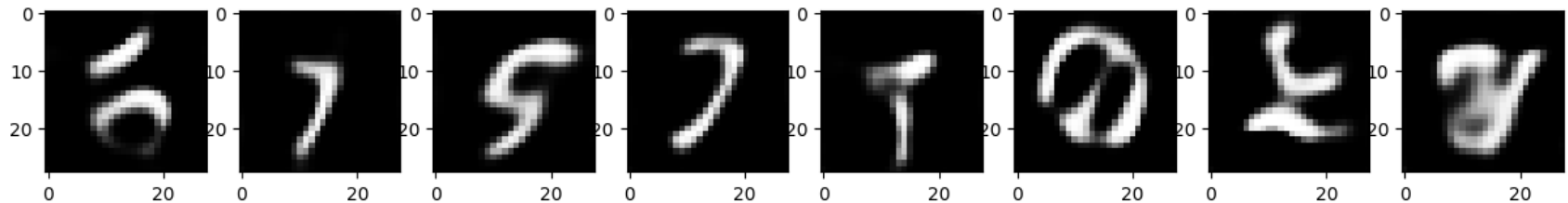
10% | 299/3000 [00:22<03:13, 13.96it/s]
Discriminator Loss: -3.2966 | Generator Loss: 2.6900



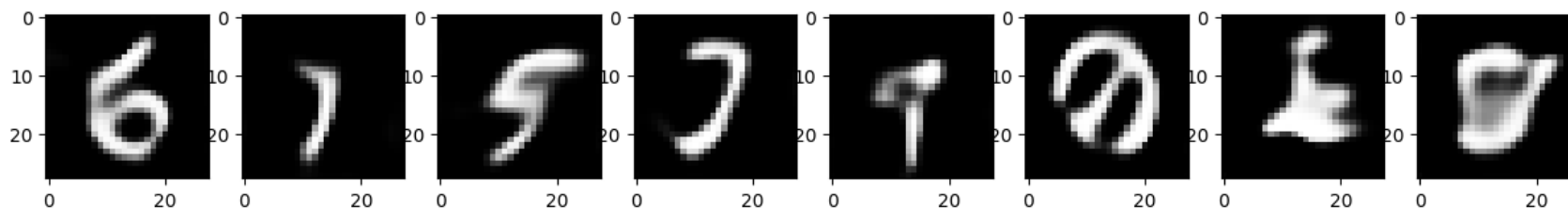
13% | 399/3000 [00:30<03:11, 13.60it/s]
Discriminator Loss: -2.5654 | Generator Loss: -0.4378



17% | 499/3000 [00:38<02:58, 14.02it/s]
Discriminator Loss: -2.7610 | Generator Loss: 0.3198

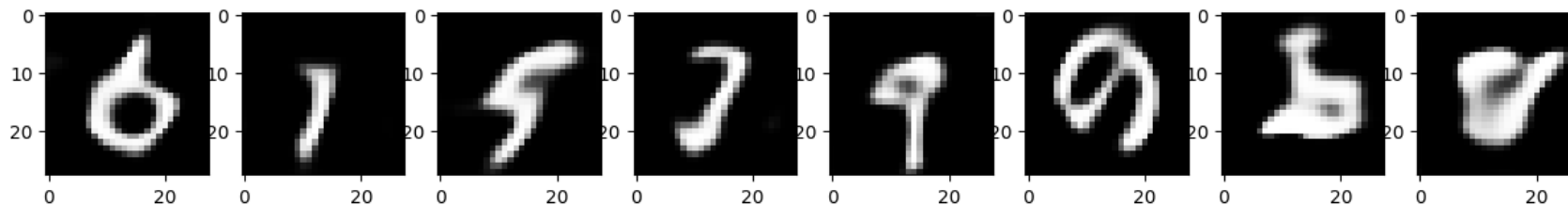


20% | 599/3000 [00:45<02:52, 13.88it/s]
Discriminator Loss: -2.0172 | Generator Loss: -0.4768



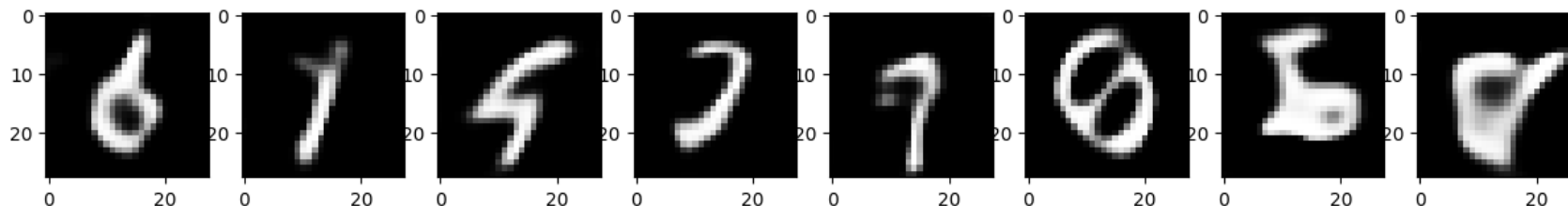
23% |  | 699/3000 [00:53<02:50, 13.47it/s]

Discriminator Loss: -2.4510 | Generator Loss: -1.8248



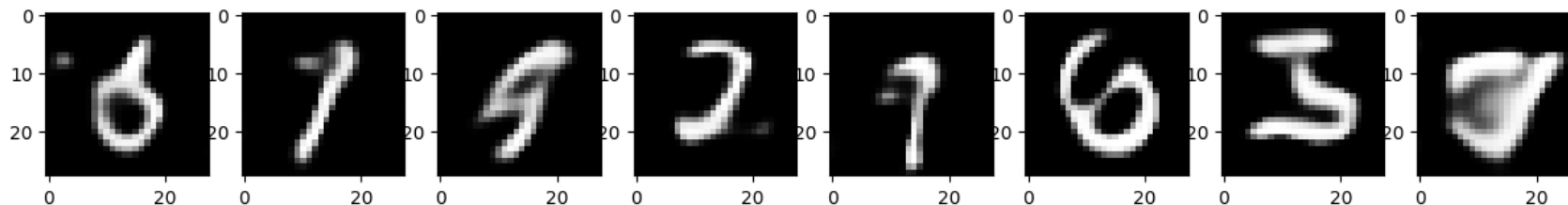
27% |  | 799/3000 [01:00<02:44, 13.34it/s]

Discriminator Loss: -2.3645 | Generator Loss: -0.3404



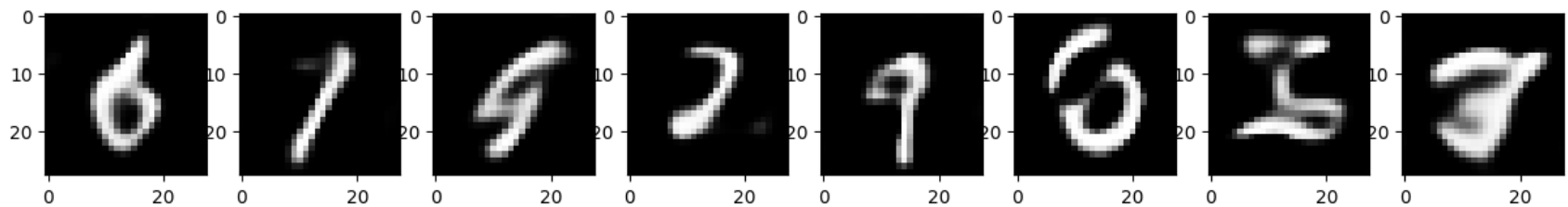
30% |  | 899/3000 [01:08<02:33, 13.68it/s]

Discriminator Loss: -2.4796 | Generator Loss: 0.0021



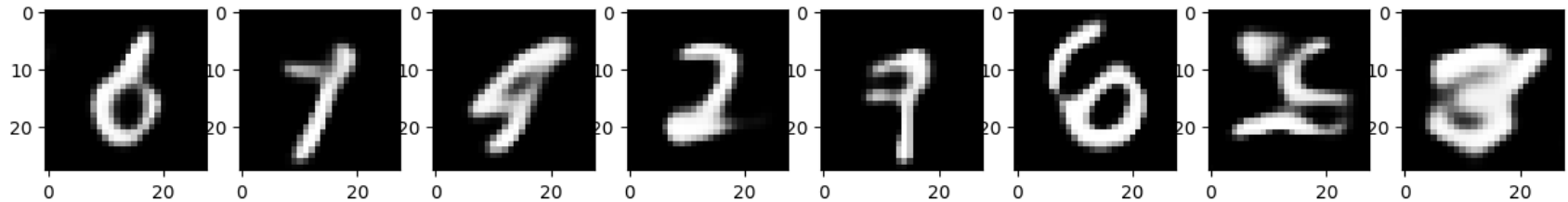
33% |  | 999/3000 [01:15<02:27, 13.54it/s]

Discriminator Loss: -2.1124 | Generator Loss: 0.7813



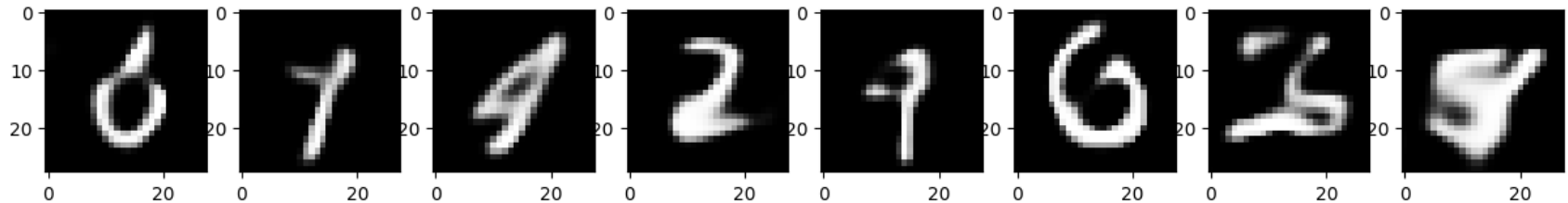
37% | ████████ | 1099/3000 [01:23<02:15, 14.03it/s]

Discriminator Loss: -2.1688 | Generator Loss: -1.2823



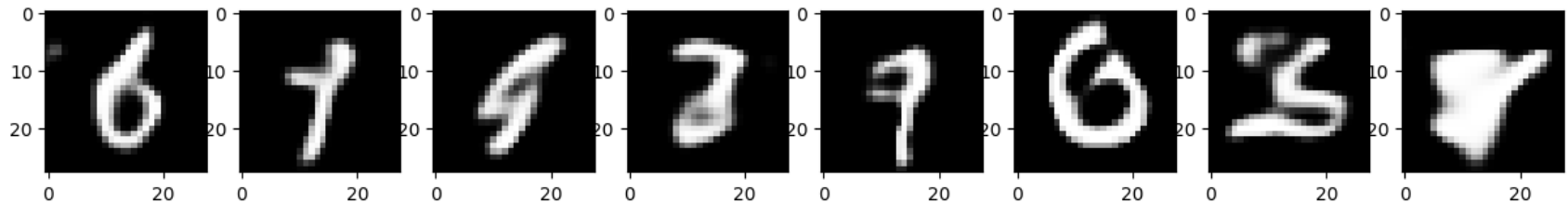
40% | ████████ | 1199/3000 [01:31<02:04, 14.47it/s]

Discriminator Loss: -2.6162 | Generator Loss: 0.8144



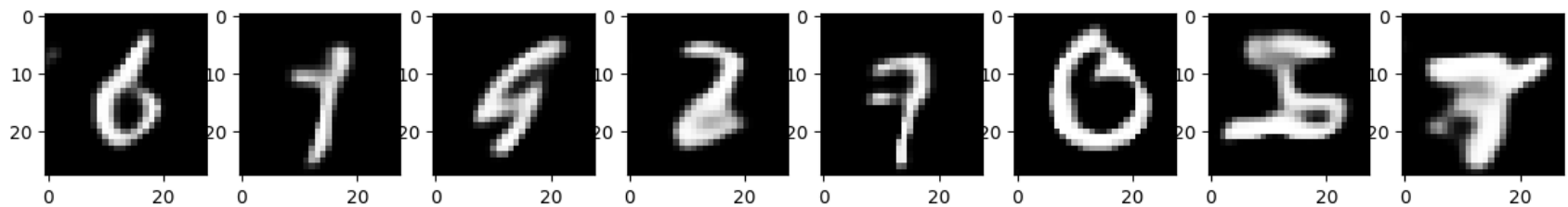
43% | ████████ | 1299/3000 [01:38<02:01, 14.00it/s]

Discriminator Loss: -1.6131 | Generator Loss: -1.5505

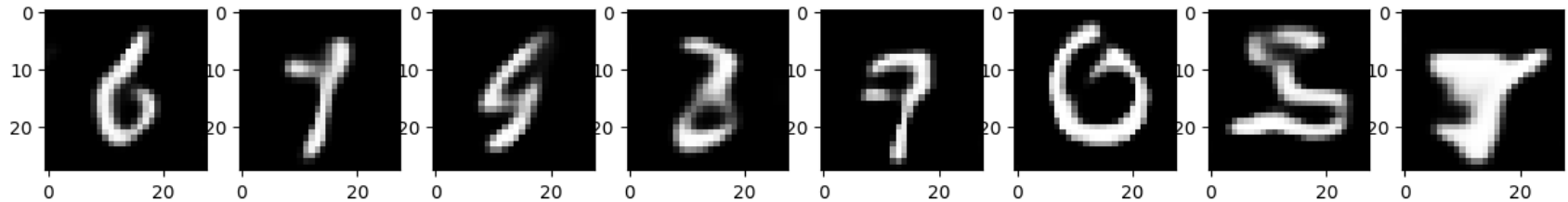


47% | ████████ | 1399/3000 [01:45<01:55, 13.89it/s]

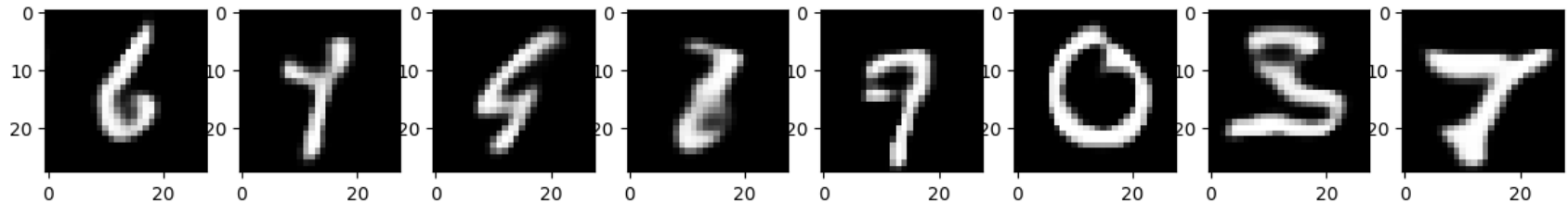
Discriminator Loss: -2.2806 | Generator Loss: -0.6631



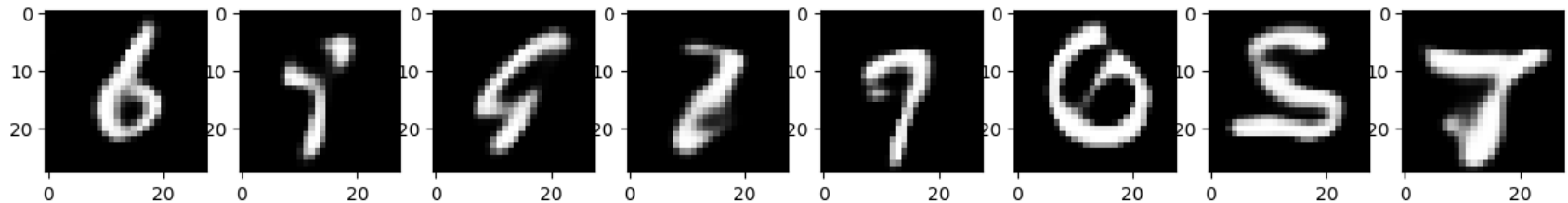
50%|██████| 1499/3000 [01:53<01:47, 13.96it/s]
Discriminator Loss: -1.8144 | Generator Loss: 0.0884



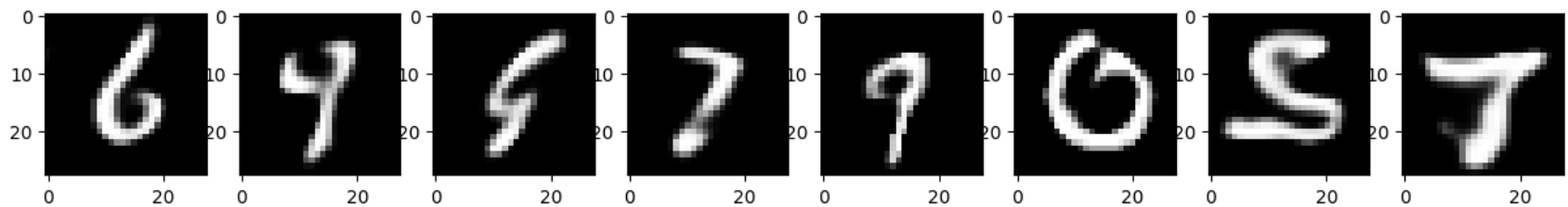
53%|██████| 1599/3000 [02:00<01:40, 13.95it/s]
Discriminator Loss: -2.4176 | Generator Loss: 0.5187



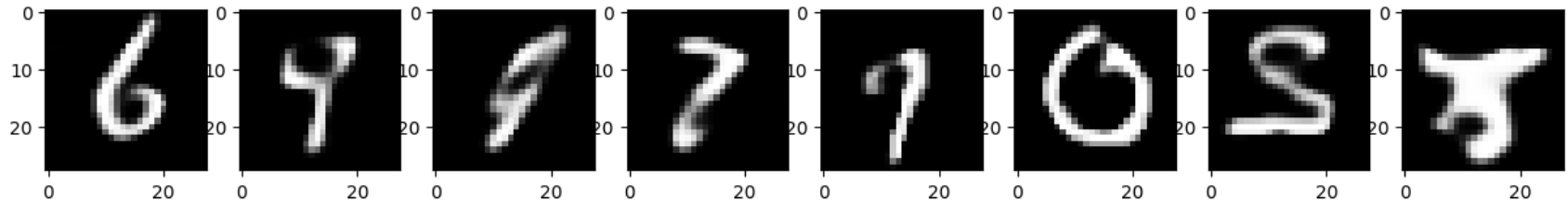
57%|██████| 1699/3000 [02:08<01:39, 13.03it/s]
Discriminator Loss: -2.1503 | Generator Loss: -0.7534



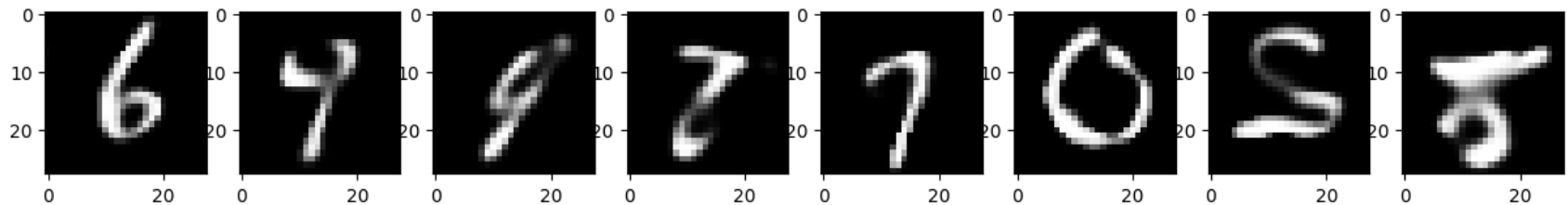
60%|██████| 1799/3000 [02:16<01:32, 12.93it/s]
Discriminator Loss: -1.8508 | Generator Loss: 1.1082



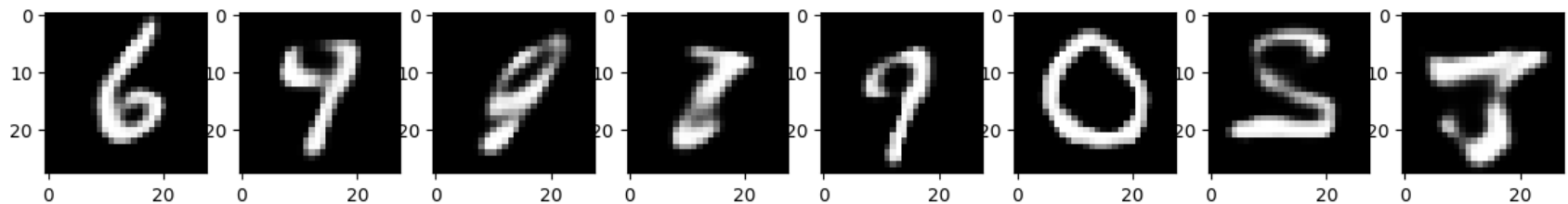
63% | ██████████ | 1899/3000 [02:24<01:24, 13.10it/s]
Discriminator Loss: -1.7729 | Generator Loss: 0.8332



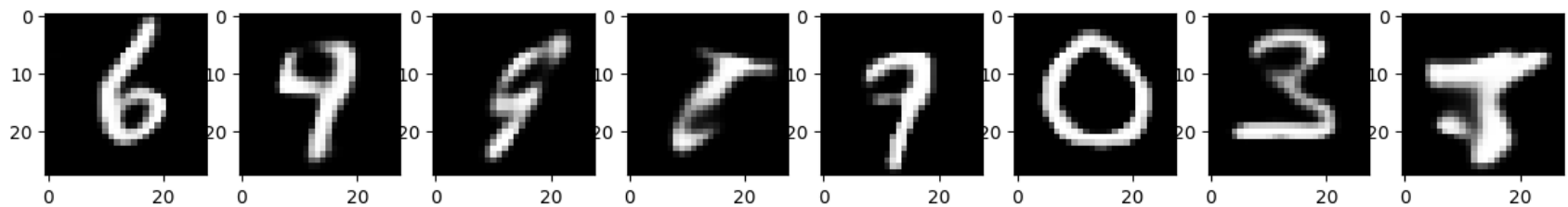
67% | ██████████ | 1999/3000 [02:32<01:12, 13.76it/s]
Discriminator Loss: -2.2643 | Generator Loss: -0.0359



70% | ██████████ | 2099/3000 [02:40<01:14, 12.17it/s]
Discriminator Loss: -2.5467 | Generator Loss: -0.6426

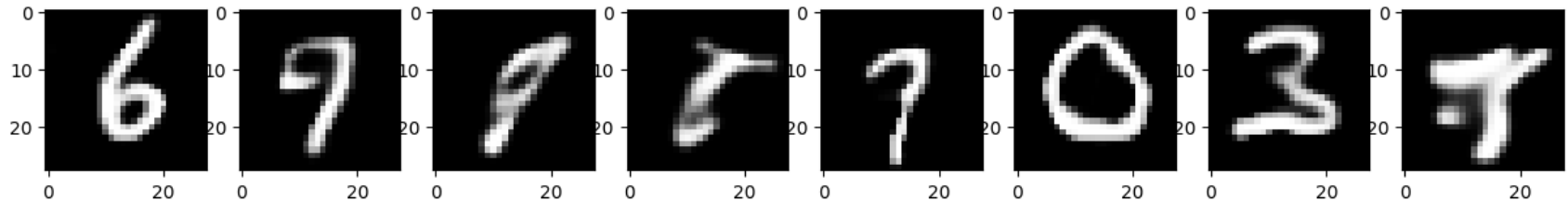


73% | ██████████ | 2199/3000 [02:48<01:01, 13.07it/s]
Discriminator Loss: -2.2032 | Generator Loss: -0.6851



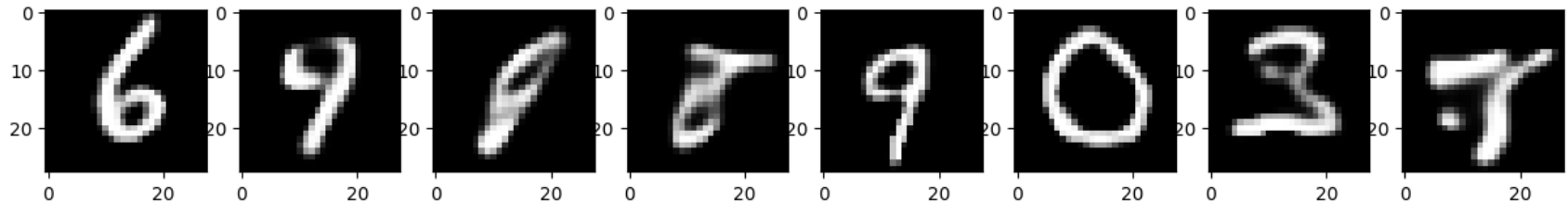
77% | ██████████ | 2299/3000 [02:56<00:53, 13.05it/s]

Discriminator Loss: -2.0186 | Generator Loss: -1.2360



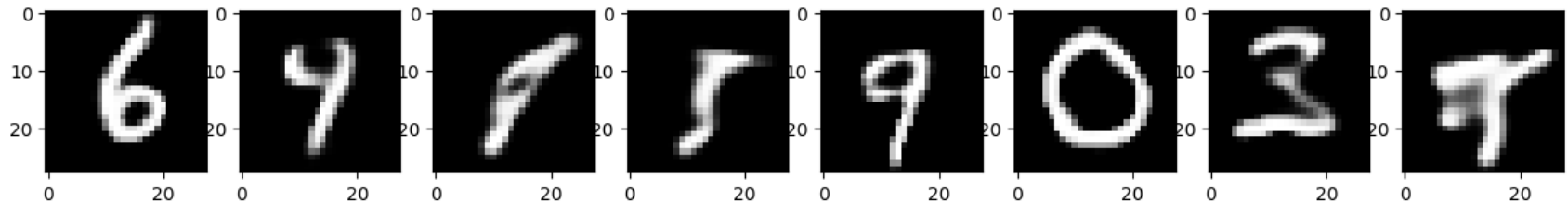
80% | ██████████ | 2399/3000 [03:04<00:46, 12.92it/s]

Discriminator Loss: -1.5457 | Generator Loss: -0.2624



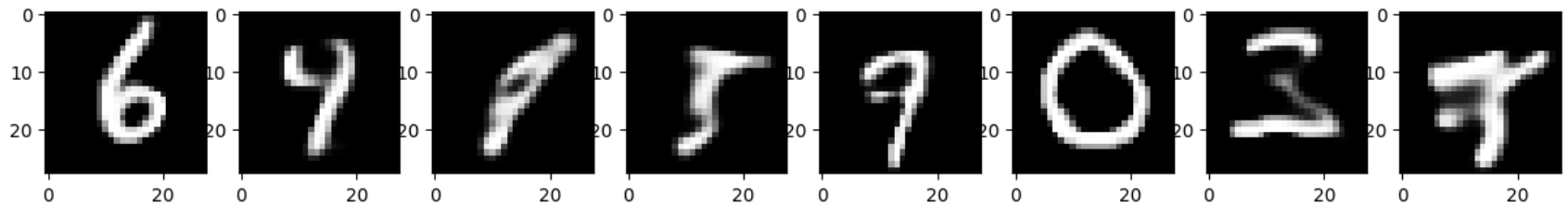
83% | ██████████ | 2499/3000 [03:12<00:41, 11.97it/s]

Discriminator Loss: -1.8485 | Generator Loss: 0.2452



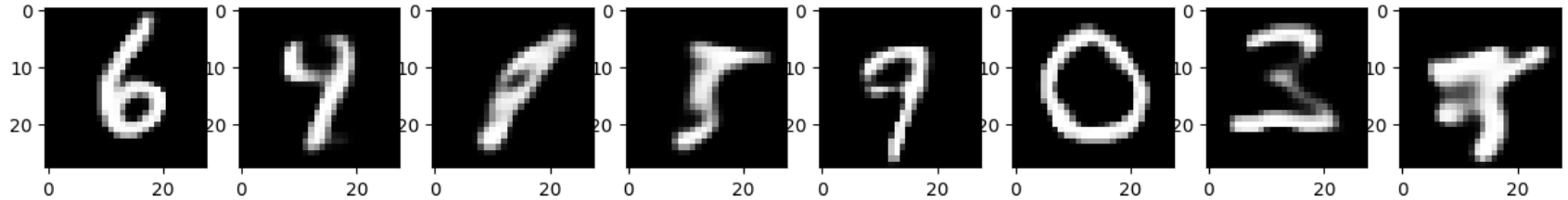
87% | ██████████ | 2599/3000 [03:20<00:30, 13.10it/s]

Discriminator Loss: -1.6915 | Generator Loss: -0.7878



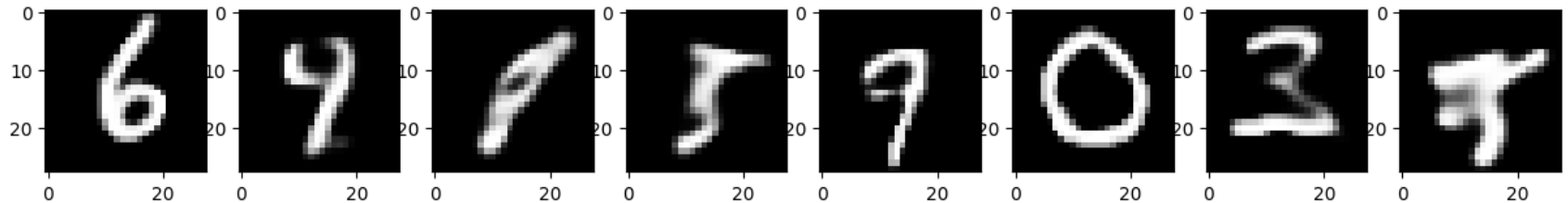
90% | ██████████ | 2699/3000 [03:29<00:23, 13.09it/s]

Discriminator Loss: -1.6973 | Generator Loss: -0.9164



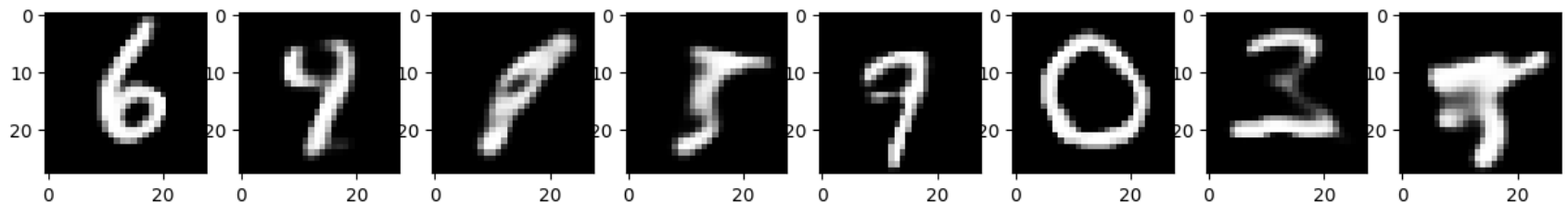
93% | ██████████ | 2799/3000 [03:37<00:15, 12.97it/s]

Discriminator Loss: -1.7472 | Generator Loss: -0.7201



97% | ██████████ | 2899/3000 [03:45<00:07, 12.75it/s]

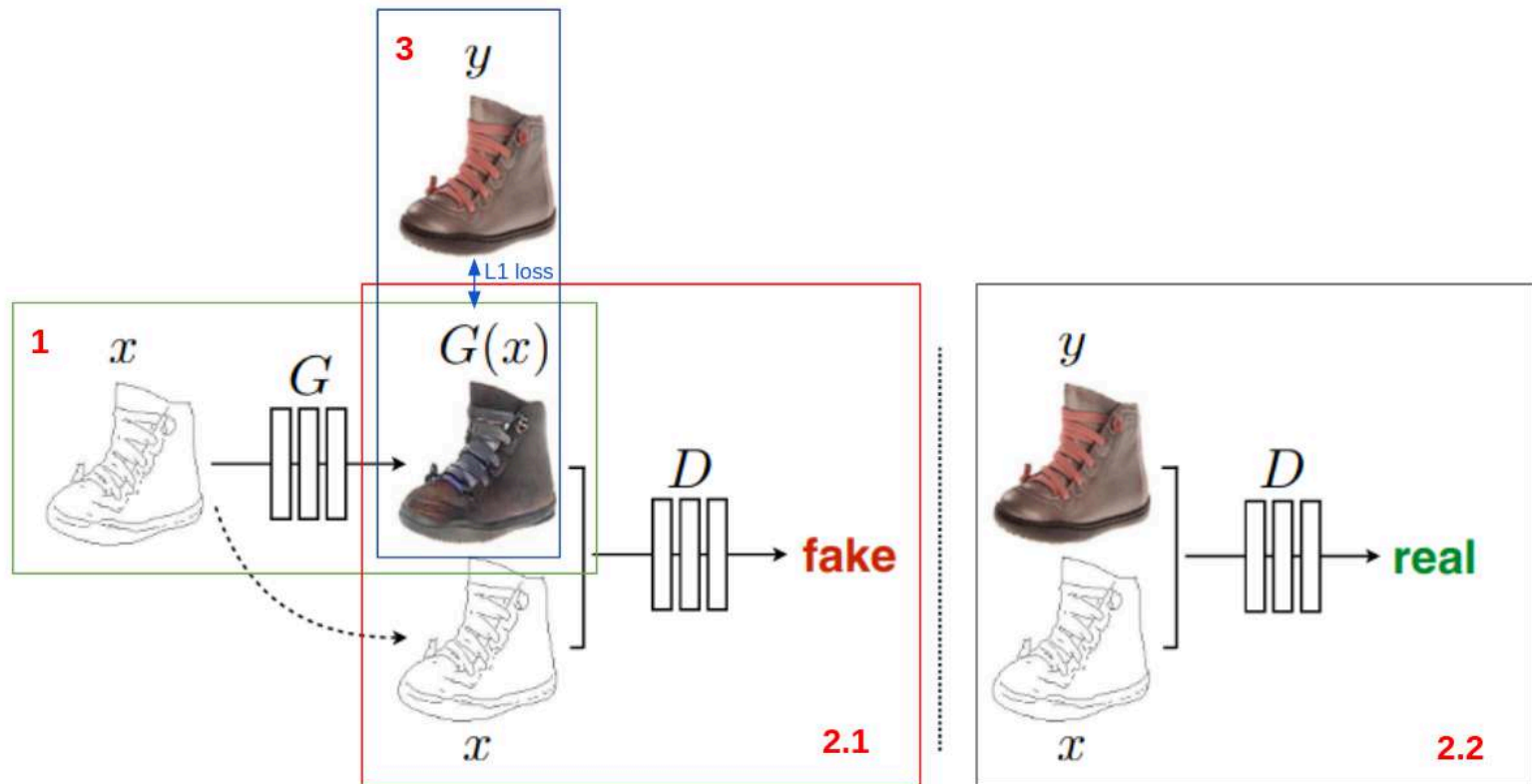
Discriminator Loss: -1.9946 | Generator Loss: -0.7606



100% | ██████████ | 3000/3000 [03:53<00:00, 12.85it/s]

Part 2 : pix2pix reimplementation (cGAN on paired image translation)

In this exercise, we are reimplementing a paired image translation model, an application of a generative adversarial network (GAN). The model we are going to implement is pix2pix (<https://arxiv.org/pdf/1611.07004.pdf>), one of the earliest paired image translation models based on GAN. The pipeline of pix2pix is shown in the Figure below.



From the figure above, the pipeline consists of three main parts:

- 1. Generation phase : the generator G create the generated image $G(x)$ from the given input x .
- 2. Discrimination phase :

In step 2.1, the discriminator D receives an input image x and the generated image $G(x)$, then the discriminator has to learn to predict that the generated image $G(x)$ is fake.

In step 2.2, the discriminator D receives an input image x and the ground truth image y , then the discriminator has to learn to predict that the image y is real.

- 3. Refinement phase: Refine the quality of the generated image $G(x)$ by encouraging the generated image to be close to an actual image y by using L1 as an objective.

The objective of pix2pix is to train an optimal generator G^* based on the objective function :

$$G^* = \operatorname{argmin}_G \max_D L_{cGAN}(G, D) + \lambda L_1(G)$$

- The term $\operatorname{argmin}_G \max_D L_{cGAN}(G, D)$ is the objective function of the first and second step, which is a standard cGAN loss : $L_{cGAN}(G, D) = E_{x,y}[\log D(x, y)] + E_{x,z}[\log(1 - D(x, G(x, z)))]$. The noise z is embedded in the generator in the form of dropout.
- The term $L_1(G)$ is the objective function of the third step where $L_1(G) = E_{x,y,z}[\|y - G(x, z)\|_1]$

The subsections will explain the dataset and training setup of this exercise.

Get dataset

```
In [22]: !wget http://efrosgans.eecs.berkeley.edu/pix2pix/datasets/facades.tar.gz
         !tar -xzf facades.tar.gz
```

```
--2026-03-25 21:02:20-- http://efrosgans.eecs.berkeley.edu/pix2pix/datasets/facades.tar.gz
Resolving efrosgans.eecs.berkeley.edu (efrosgans.eecs.berkeley.edu)... 128.32.244.190
Connecting to efrosgans.eecs.berkeley.edu (efrosgans.eecs.berkeley.edu)|128.32.244.190|:80... connected.
HTTP request sent, awaiting response... 200 OK
Length: 30168306 (29M) [application/x-gzip]
Saving to: 'facades.tar.gz'
```

```
facades.tar.gz      100%[=====>]  28.77M   701KB/s   in 39s
```

```
2026-03-25 21:03:00 (746 KB/s) - 'facades.tar.gz' saved [30168306/30168306]
```

Import library

```
In [1]: import cv2
        import glob
        import numpy as np
        import torch
```

```
import torch.nn.functional as F
from torch import nn
from torchvision import transforms
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec
```

Setting up facade dataset

The dataset chosen for this exercise is the CMP Facade Database which is a pair of facade images and its segmented component stored in RGB value. The objective of this exercise is to generate a facade given its simplified segmented component. Both input and output is a 256 x 256 RGB image normalized to $[-1, 1]$.

```
In [2]: train = (np.array([cv2.imread(i) for i in glob.glob('facades/train/*')], dtype = np.float32) / 255).transpose(2, 0, 1)
train = (train - 0.5) * 2 #shift from [0,1] to [-1, 1]
trainX = train[:, :, :, 256:]
trainY = train[:, :, :, :256]

val = (np.array([cv2.imread(i) for i in glob.glob('facades/val/*')], dtype = np.float32) / 255).transpose(2, 0, 1)
val = (val - 0.5) * 2 #shift from [0,1] to [-1, 1]
valX = val[:, :, :, 256:]
valY = val[:, :, :, :256]

print("Input size : {}, Output size = {}".format(trainX.shape, trainY.shape))
```

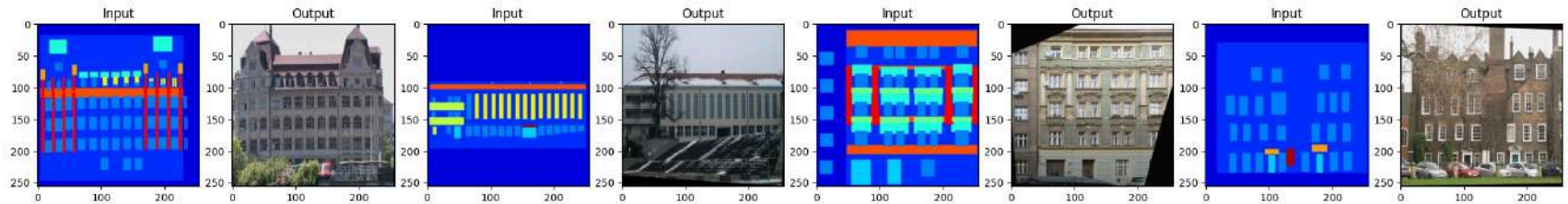
Input size : (400, 3, 256, 256), Output size = (400, 3, 256, 256)

Dataset Visualization

```
In [3]: import matplotlib.pyplot as plt
import numpy as np
plt.figure(figsize = (30,90))
for i in range(4):
    idx = np.random.randint(len(trainX))

    plt.subplot( int('19{}'.format(2*i+1)) )
    plt.title('Input')
    plt.imshow( (0.5 * trainX[idx].transpose((1, 2, 0)) + 0.5)[..., ::-1] , cmap = 'gray' )
    plt.subplot( int('19{}'.format(2*i+2)) )
```

```
plt.title('Output')
plt.imshow( (0.5 * trainY[idx].transpose((1, 2, 0)) + 0.5)[..., ::-1], cmap = 'gray' )
plt.show()
```

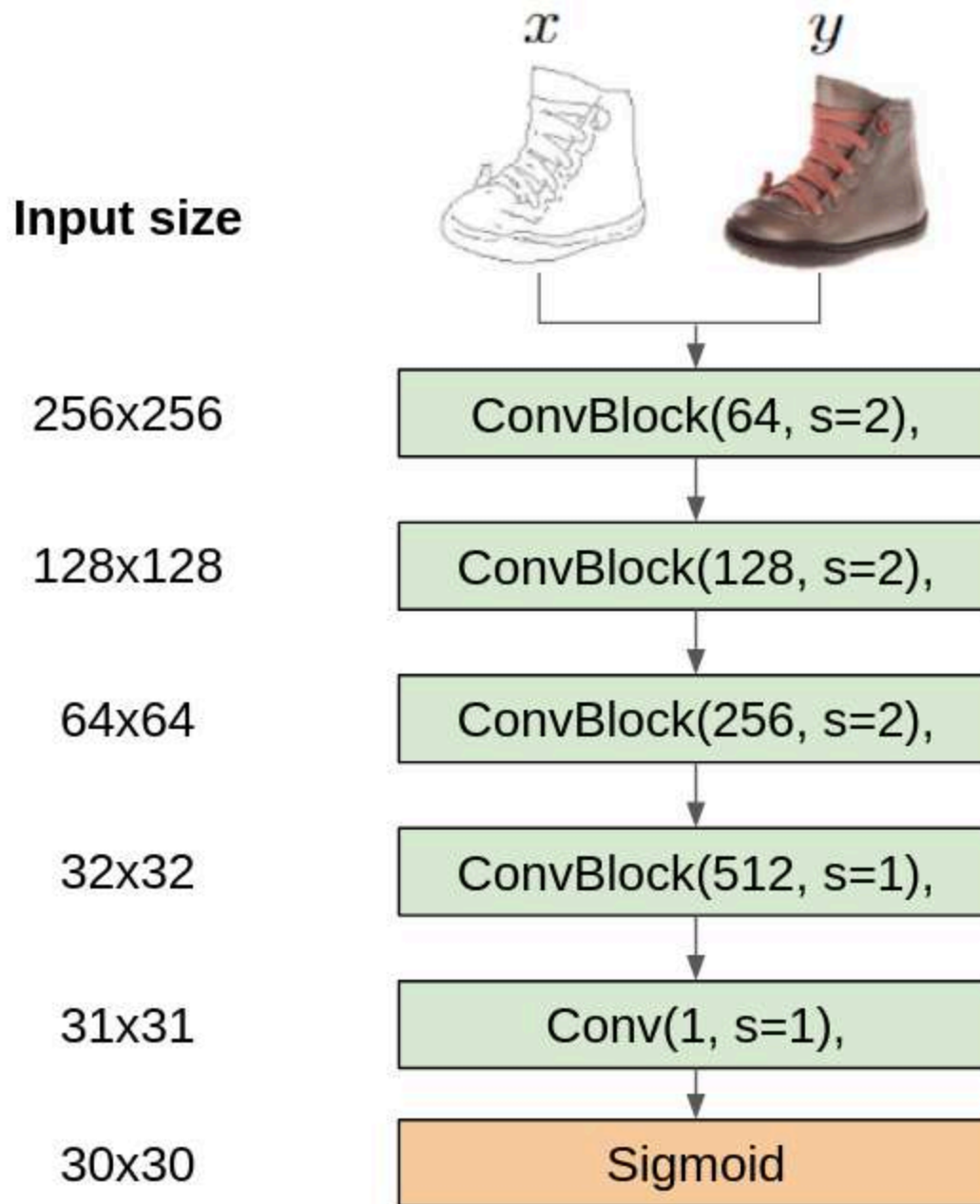


Note

If you have trouble understanding the instruction provided in this homework or have any ambiguity about the instruction, you could also read the appendix section (section 6.1-6.2) in the paper for a detailed explanation.

Discriminator network

In this section, we are going to implement a discriminator network of pix2pix. The description of the discriminator network is provided in the Figure below.



The network also has the following specific requirements:

- All convolutions are 4×4 spatial filters
- ConvBlock is a Convolution-InstanceNorm-ReLU layer
- InstanceNorm is not applied to the first C64 layer
- All ReLUs are leaky, with a slope of 0.2

TODO 11: Implement the discriminator network based on the description above.

TODO 12: What should be the size of the input and output of the discriminator for this task? Verify that the input and output of the implemented network are the same as the answer you have provided.

```
In [4]: from torchinfo import summary
```

```
In [5]: class Discriminator(nn.Module):
    #TODO11 implement the discriminator network
    def __init__(self, in_channels=3):
        super().__init__()

        def discrim_block(in_filters, out_filters, stride, normalization=True):
            layers = [nn.Conv2d(in_filters, out_filters, kernel_size=4, stride=stride, padding=1)]
            if normalization:
                layers.append(nn.InstanceNorm2d(out_filters))
            layers.append(nn.LeakyReLU(0.2, inplace=True))
            return layers

        self.conv = nn.Sequential(
            *discrim_block(in_channels * 2, 64, stride=2, normalization=False),
            *discrim_block(64, 128, stride=2),
            *discrim_block(128, 256, stride=2),
            *discrim_block(256, 512, stride=1),
            nn.Conv2d(512, 1, kernel_size=4, stride=1, padding=1, bias=False),
            nn.Sigmoid()
        )
        def forward(self, img_A, img_B):
            img = torch.cat((img_A, img_B), 1)
            return self.conv(img)

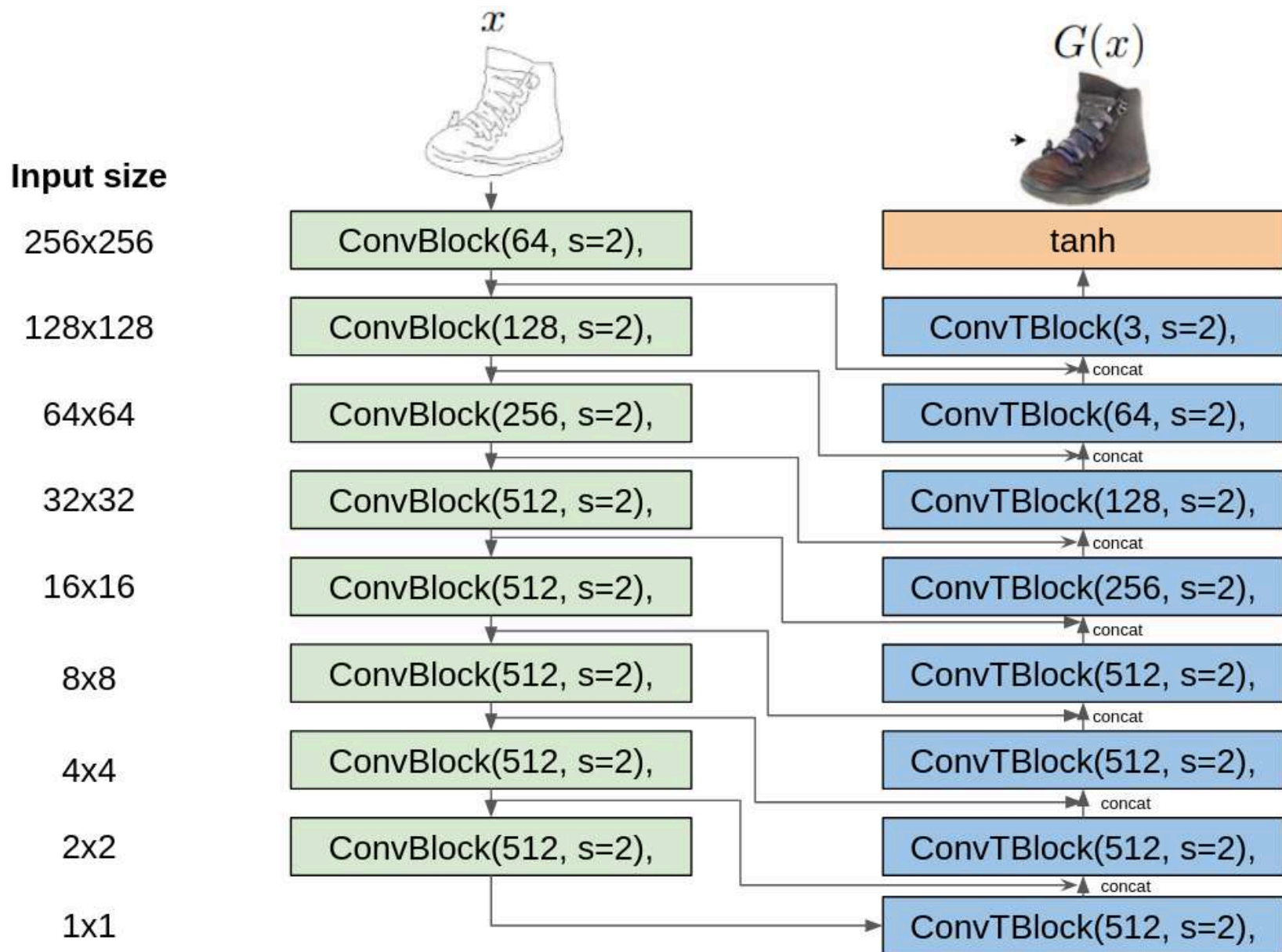
discriminator = Discriminator().cuda()
```

```
In [6]: #TODO12 verify the discriminator
print(summary(discriminator, input_size=[(400, 3, 256, 256), (400, 3, 256, 256)]))
```

```
=====
Layer (type:depth-idx)                   Output Shape                  Param #
=====
Discriminator                           [400, 1, 30, 30]             --
└─Sequential: 1-1                       [400, 1, 30, 30]             --
    └─Conv2d: 2-1                        [400, 64, 128, 128]          6,208
        └─LeakyReLU: 2-2                 [400, 64, 128, 128]          --
            └─Conv2d: 2-3                 [400, 128, 64, 64]           131,200
                └─InstanceNorm2d: 2-4     [400, 128, 64, 64]           --
                    └─LeakyReLU: 2-5      [400, 128, 64, 64]           --
                        └─Conv2d: 2-6      [400, 256, 32, 32]           524,544
                            └─InstanceNorm2d: 2-7 [400, 256, 32, 32]           --
                                └─LeakyReLU: 2-8 [400, 256, 32, 32]           --
                                    └─Conv2d: 2-9 [400, 512, 31, 31]           2,097,664
                                        └─InstanceNorm2d: 2-10 [400, 512, 31, 31]           --
                                            └─LeakyReLU: 2-11 [400, 512, 31, 31]           --
                                                └─Conv2d: 2-12 [400, 1, 30, 30]             8,192
                                                    └─Sigmoid: 2-13 [400, 1, 30, 30]             --
=====
Total params: 2,767,808
Trainable params: 2,767,808
Non-trainable params: 0
Total mult-adds (Units.TERABYTES): 1.28
=====
Input size (MB): 629.15
Forward/backward pass size (MB): 7449.41
Params size (MB): 11.07
Estimated Total Size (MB): 8089.62
=====
```

Generator network

In this section, we are going to implement a generator network of pix2pix. The generator is based on the U-NET based architecture (<https://arxiv.org/abs/1505.04597>). The Description of the generator network is provided in the Figure below.



The network also has the following specific requirements:

- All convolutions are 4×4 spatial filters
- ConvBlock is a Convolution-InstanceNorm-ReLU layer

- ConvTBlock is a ConvolutionTranspose-InstanceNorm-DropOut-ReLU layer with a dropout rate of 50%
- InstanceNorm is not applied to the first C64 layer in the encoder
- All ReLUs in the encoder are leaky, with a slope of 0.2, while ReLUs in the decoder are not leaky

TODO 13: Implement the generator network based on the description above.

TODO 14: What should be the size of the input and output of the generator for this task? Verify that the input and output of the implemented network are the same as the answer you have provided.

In [7]: *#HINT : you could also put multiple layers in a single list using nn.ModuleList*

```
class Example(nn.Module):
    def __init__(self):
        super().__init__()
        self.convs = nn.ModuleList([nn.Conv2d(25, 25, 3) for i in range(5)])
    def forward(self, x):
        for i in range(len(self.convs)):
            x = self.convs[i](x)
        return x
ex = Example().cuda()
print(ex(torch.zeros((8, 25, 32, 32)).cuda()).shape)
```

torch.Size([8, 25, 22, 22])

```
In [8]: class UNetDown(nn.Module):
    def __init__(self, in_size, out_size, normalize=True):
        super(UNetDown, self).__init__()
        layers = [nn.Conv2d(in_size, out_size, kernel_size=4, stride=2, padding=1, bias=False)]
        if normalize:
            layers.append(nn.InstanceNorm2d(out_size))
        layers.append(nn.LeakyReLU(0.2))

        self.model = nn.Sequential(*layers)

    def forward(self, x):
        return self.model(x)

class UNetUp(nn.Module):
    def __init__(self, in_size, out_size, dropout=0.0):
        super(UNetUp, self).__init__()
        layers = [
```

```

        nn.ConvTranspose2d(in_size, out_size, kernel_size=4, stride=2, padding=1, bias=False),
        nn.InstanceNorm2d(out_size),
        nn.ReLU(inplace=True)
    ]
    if dropout:
        layers.append(nn.Dropout(dropout))
    self.model = nn.Sequential(*layers)

    def forward(self, x, skip_input):
        x = self.model(x)
        return torch.cat((x, skip_input), 1)

```

```

In [9]: class GeneratorUNet(nn.Module):
    def __init__(self, in_channels=3, out_channels=3):
        #TODO13 implement the generator network
        super(GeneratorUNet, self).__init__()

        self.down1 = UNetDown(in_channels, 64, normalize=False)
        self.down2 = UNetDown(64, 128)
        self.down3 = UNetDown(128, 256)
        self.down4 = UNetDown(256, 512)
        self.down5 = UNetDown(512, 512)
        self.down6 = UNetDown(512, 512)
        self.down7 = UNetDown(512, 512)
        self.down8 = UNetDown(512, 512, normalize=False)

        self.up1 = UNetUp(512, 512, dropout=0.5)
        self.up2 = UNetUp(1024, 512, dropout=0.5)
        self.up3 = UNetUp(1024, 512, dropout=0.5)
        self.up4 = UNetUp(1024, 512, dropout=0.5)
        self.up5 = UNetUp(1024, 256, dropout=0.5)
        self.up6 = UNetUp(512, 128, dropout=0.5)
        self.up7 = UNetUp(256, 64, dropout=0.5)

        self.final = nn.Sequential(
            nn.ConvTranspose2d(128, out_channels, 4, 2, 1),
            nn.Tanh()
        )

    def forward(self, x):
        d1 = self.down1(x)
        d2 = self.down2(d1)

```

```
d3 = self.down3(d2)
d4 = self.down4(d3)
d5 = self.down5(d4)
d6 = self.down6(d5)
d7 = self.down7(d6)
d8 = self.down8(d7)
u1 = self.up1(d8, d7)
u2 = self.up2(u1, d6)
u3 = self.up3(u2, d5)
u4 = self.up4(u3, d4)
u5 = self.up5(u4, d3)
u6 = self.up6(u5, d2)
u7 = self.up7(u6, d1)
return self.final(u7)
```

```
generator = GeneratorUNet().cuda()
```

```
In [10]: #TODO14 verify the generator
print(summary(generator, input_size=(1, 3, 256, 256)))
```

Layer (type:depth-idx)	Output Shape	Param #
GeneratorUNet	[1, 3, 256, 256]	--
└UNetDown: 1-1	[1, 64, 128, 128]	--
└Sequential: 2-1	[1, 64, 128, 128]	--
└Conv2d: 3-1	[1, 64, 128, 128]	3,072
└LeakyReLU: 3-2	[1, 64, 128, 128]	--
└UNetDown: 1-2	[1, 128, 64, 64]	--
└Sequential: 2-2	[1, 128, 64, 64]	--
└Conv2d: 3-3	[1, 128, 64, 64]	131,072
└InstanceNorm2d: 3-4	[1, 128, 64, 64]	--
└LeakyReLU: 3-5	[1, 128, 64, 64]	--
└UNetDown: 1-3	[1, 256, 32, 32]	--
└Sequential: 2-3	[1, 256, 32, 32]	--
└Conv2d: 3-6	[1, 256, 32, 32]	524,288
└InstanceNorm2d: 3-7	[1, 256, 32, 32]	--
└LeakyReLU: 3-8	[1, 256, 32, 32]	--
└UNetDown: 1-4	[1, 512, 16, 16]	--
└Sequential: 2-4	[1, 512, 16, 16]	--
└Conv2d: 3-9	[1, 512, 16, 16]	2,097,152
└InstanceNorm2d: 3-10	[1, 512, 16, 16]	--
└LeakyReLU: 3-11	[1, 512, 16, 16]	--
└UNetDown: 1-5	[1, 512, 8, 8]	--
└Sequential: 2-5	[1, 512, 8, 8]	--
└Conv2d: 3-12	[1, 512, 8, 8]	4,194,304
└InstanceNorm2d: 3-13	[1, 512, 8, 8]	--
└LeakyReLU: 3-14	[1, 512, 8, 8]	--
└UNetDown: 1-6	[1, 512, 4, 4]	--
└Sequential: 2-6	[1, 512, 4, 4]	--
└Conv2d: 3-15	[1, 512, 4, 4]	4,194,304
└InstanceNorm2d: 3-16	[1, 512, 4, 4]	--
└LeakyReLU: 3-17	[1, 512, 4, 4]	--
└UNetDown: 1-7	[1, 512, 2, 2]	--
└Sequential: 2-7	[1, 512, 2, 2]	--
└Conv2d: 3-18	[1, 512, 2, 2]	4,194,304
└InstanceNorm2d: 3-19	[1, 512, 2, 2]	--
└LeakyReLU: 3-20	[1, 512, 2, 2]	--
└UNetDown: 1-8	[1, 512, 1, 1]	--
└Sequential: 2-8	[1, 512, 1, 1]	--
└Conv2d: 3-21	[1, 512, 1, 1]	4,194,304
└LeakyReLU: 3-22	[1, 512, 1, 1]	--

UNetUp: 1-9	[1, 1024, 2, 2]	--
└─Sequential: 2-9	[1, 512, 2, 2]	--
└─ConvTranspose2d: 3-23	[1, 512, 2, 2]	4,194,304
└─InstanceNorm2d: 3-24	[1, 512, 2, 2]	--
└─ReLU: 3-25	[1, 512, 2, 2]	--
└─Dropout: 3-26	[1, 512, 2, 2]	--
UNetUp: 1-10	[1, 1024, 4, 4]	--
└─Sequential: 2-10	[1, 512, 4, 4]	--
└─ConvTranspose2d: 3-27	[1, 512, 4, 4]	8,388,608
└─InstanceNorm2d: 3-28	[1, 512, 4, 4]	--
└─ReLU: 3-29	[1, 512, 4, 4]	--
└─Dropout: 3-30	[1, 512, 4, 4]	--
UNetUp: 1-11	[1, 1024, 8, 8]	--
└─Sequential: 2-11	[1, 512, 8, 8]	--
└─ConvTranspose2d: 3-31	[1, 512, 8, 8]	8,388,608
└─InstanceNorm2d: 3-32	[1, 512, 8, 8]	--
└─ReLU: 3-33	[1, 512, 8, 8]	--
└─Dropout: 3-34	[1, 512, 8, 8]	--
UNetUp: 1-12	[1, 1024, 16, 16]	--
└─Sequential: 2-12	[1, 512, 16, 16]	--
└─ConvTranspose2d: 3-35	[1, 512, 16, 16]	8,388,608
└─InstanceNorm2d: 3-36	[1, 512, 16, 16]	--
└─ReLU: 3-37	[1, 512, 16, 16]	--
└─Dropout: 3-38	[1, 512, 16, 16]	--
UNetUp: 1-13	[1, 512, 32, 32]	--
└─Sequential: 2-13	[1, 256, 32, 32]	--
└─ConvTranspose2d: 3-39	[1, 256, 32, 32]	4,194,304
└─InstanceNorm2d: 3-40	[1, 256, 32, 32]	--
└─ReLU: 3-41	[1, 256, 32, 32]	--
└─Dropout: 3-42	[1, 256, 32, 32]	--
UNetUp: 1-14	[1, 256, 64, 64]	--
└─Sequential: 2-14	[1, 128, 64, 64]	--
└─ConvTranspose2d: 3-43	[1, 128, 64, 64]	1,048,576
└─InstanceNorm2d: 3-44	[1, 128, 64, 64]	--
└─ReLU: 3-45	[1, 128, 64, 64]	--
└─Dropout: 3-46	[1, 128, 64, 64]	--
UNetUp: 1-15	[1, 128, 128, 128]	--
└─Sequential: 2-15	[1, 64, 128, 128]	--
└─ConvTranspose2d: 3-47	[1, 64, 128, 128]	262,144
└─InstanceNorm2d: 3-48	[1, 64, 128, 128]	--
└─ReLU: 3-49	[1, 64, 128, 128]	--
└─Dropout: 3-50	[1, 64, 128, 128]	--

```

└─Sequential: 1-16          [1, 3, 256, 256]      --
    └─ConvTranspose2d: 2-16  [1, 3, 256, 256]      6,147
    └─Tanh: 2-17             [1, 3, 256, 256]      --

```

```

=====
Total params: 54,404,099
Trainable params: 54,404,099
Non-trainable params: 0
Total mult-adds (Units.GIGABYTES): 18.14
=====

```

```

=====
Input size (MB): 0.79
Forward/backward pass size (MB): 33.72
Params size (MB): 217.62
Estimated Total Size (MB): 252.13
=====

```

Data preparation

After the model is initialized, we then create a dataloader to sample the training data. In this paper, to sample the training data, you have to sequentially perform the following steps :

1. Randomly sample the data from the training set
2. Resizing both input and target to 286×286 .
3. Randomly cropping back both images to size 256×256 .
4. Random mirroring the images

TODO15: Implement a dataloader based on the description above. You are allowed to use the function in torchvision.transforms (<https://pytorch.org/vision/main/transforms.html>).

```

In [11]: import torchvision.transforms.functional as TF
import random
from torch.utils.data import Dataset, DataLoader
from torchvision.transforms import RandomCrop

```

```

In [12]: BATCH_SIZE = 1

```

```

In [13]: # TODO15 implement a dataloader
class Pix2PixDataset(Dataset):
    def __init__(self, trainX, trainY):

```

```

        self.trainX = trainX
        self.trainY = trainY

    def __len__(self):
        return len(self.trainX)

    def __getitem__(self, idx):
        x = torch.tensor(self.trainX[idx], dtype=torch.float32)
        y = torch.tensor(self.trainY[idx], dtype=torch.float32)

        x = TF.resize(x, [286, 286], antialias=True)
        y = TF.resize(y, [286, 286], antialias=True)

        i, j, h, w = RandomCrop.get_params(x, output_size=(256, 256))
        x = TF.crop(x, i, j, h, w)
        y = TF.crop(y, i, j, h, w)

        if random.random() > 0.5:
            x = TF.hflip(x)
            y = TF.hflip(y)
        return {'A':x, 'B':y}

dataset = Pix2PixDataset(trainX, trainY)
dataloader = DataLoader(dataset, batch_size=BATCH_SIZE, shuffle=True)

```

Dataloader verification

TODO16: Show that the implemented dataloader is working as intended. For instance, are both input and output are flipped and cropped correctly? To obtain a full score, you have to show at least eight data points.

```

In [14]: #TODO16 show that the dataloader is working properly
data_iter = iter(dataloader)
plt.figure(figsize=(24, 6))

for i in range(8): # Show 8 data points
    batch = next(data_iter)

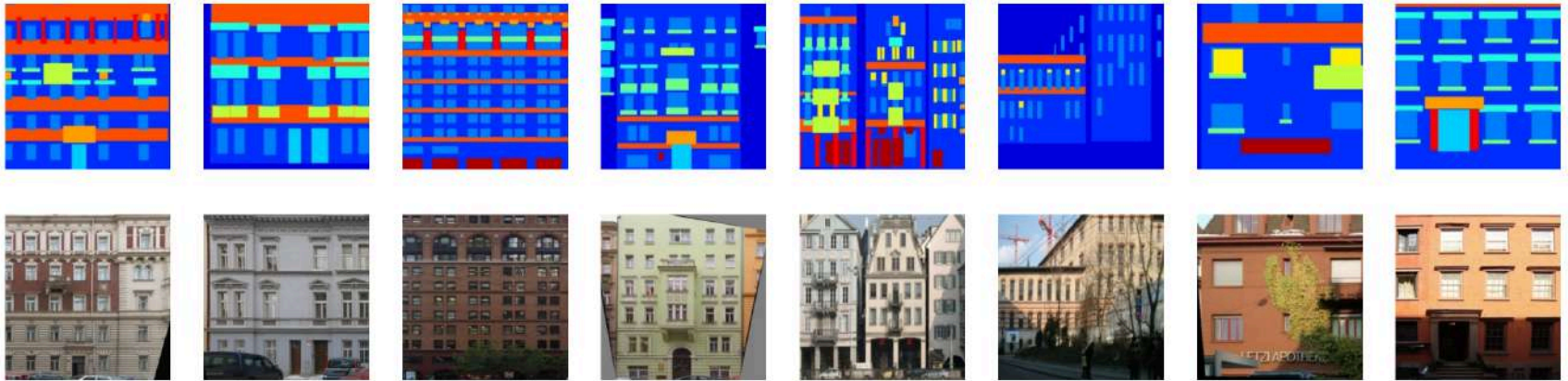
    # Plot Input A (Condition)
    ax1 = plt.subplot(2, 8, i + 1)
    img_A = 0.5 * batch['A'][0].numpy().transpose(1, 2, 0) + 0.5

```

```
plt.imshow(img_A[..., ::-1]) # [..., ::-1] is for BGR to RGB color fix
plt.axis('off')

# Plot Target B
ax2 = plt.subplot(2, 8, i + 1 + 8)
img_B = 0.5 * batch['B'][0].numpy().transpose(1, 2, 0) + 0.5
plt.imshow(img_B[..., ::-1])
plt.axis('off')

plt.show()
```



Parameter Initialization

Model hyperparameters and optimizers have already been prepared.

```
In [15]: import torch.optim as optim
from tqdm import tqdm
lr = 2e-4
LAMBDA = 100
BATCH_SIZE = 1
G_optimizer = optim.Adam(generator.parameters(), lr=lr, betas = (0.5, 0.999))
D_optimizer = optim.Adam(discriminator.parameters(), lr=lr, betas = (0.5, 0.999))
```

Training loop

The training process has the following specific requirements:

- The objective is divided by 2 while optimizing D , which slows down the rate at which D learns relative to G .
- This paper trains the generator G to maximize $\log D(x, G(x, z))$ instead of minimizing $\log(1 - D(x, G(x, z)))$ as the latter term does not provide sufficient gradient.

TODO17: Sample the data using the dataloader.

TODO18: Calculate the discriminator loss and update the discriminator.

- During the update, the loss term $\log(1 - D(x, G(x, z)))$ contains both generator and discriminator. However, we only want to update the discriminator. Therefore, you have to detach the input from the generator graph first. Read <https://pytorch.org/docs/stable/generated/torch.Tensor.detach.html> for additional detail. \

TODO19: Calculate the generator loss and update the generator.

HINT

Hint 1: If you are struggling with this part, you could also read the PyTorch DCGAN tutorial as a guideline (https://pytorch.org/tutorials/beginner/dcgan_faces_tutorial.html).

Hint 2: You could remove the L1 loss while debugging since the generator G could still generate the synthetic image even if the L1 loss is removed, though at the cost of increasing image artifacts.

Note

The training schedule in this homework is only one eighth of the original schedule. It is expected that the generated image quality is worse than the one shown in the paper. Nevertheless, the generated facade should still resemble an actual one.

```
In [16]: losses = {'D' : [None], 'G' : [None]}

criterion_GAN = nn.BCELoss() # We use BCELoss since our model already apply Tanh(-1 to 1)
criterion_pixelwise = nn.L1Loss()
valid = torch.ones(BATCH_SIZE, 1, 30, 30).cuda()
fake = torch.zeros(BATCH_SIZE, 1, 30, 30).cuda()
data_iter = iter(dataloader)

for i in tqdm(range(20001)):
    #TODO17 sample the data from the dataloader
    try:
```

```

    batch = next(data_iter)
except:
    data_iter = iter(dataloader)
    batch = next(data_iter)

real_A = batch['A'].cuda()
real_B = batch['B'].cuda()
fake_B = generator(real_A)
#TODO18 calculate the discriminator loss and update the discriminator
D_optimizer.zero_grad()

# Real Loss
pred_real = discriminator(real_A, real_B)
loss_real = criterion_GAN(pred_real, valid)

# Fake loss
pred_fake = discriminator(real_A, fake_B.detach())
loss_fake = criterion_GAN(pred_fake, fake)
loss_D = (loss_real + loss_fake) * 0.5
loss_D.backward()
D_optimizer.step()

#TODO19 calculate the generator loss and update the generator
G_optimizer.zero_grad()
# GAN Loss (does it fool D?)
pred_fake_G = discriminator(real_A, fake_B)
loss_G_GAN = criterion_GAN(pred_fake_G, valid)
# Pixel L1 Loss (does it match real B?)
loss_G_L1 = criterion_pixelwise(fake_B, real_B)
# Total Loss
loss_G = loss_G_GAN + LAMBDA * loss_G_L1
loss_G.backward()
G_optimizer.step()

losses['D'].append(loss_D.item())
losses['G'].append(loss_G.item())

# Output visualization : If your reimplementation is correct, the generated images should start resembling
if(i % 2500 == 0):
    with torch.no_grad():
        print(losses['D'][-1], losses['G'][-1])
        plt.figure(figsize = (40, 16))

```



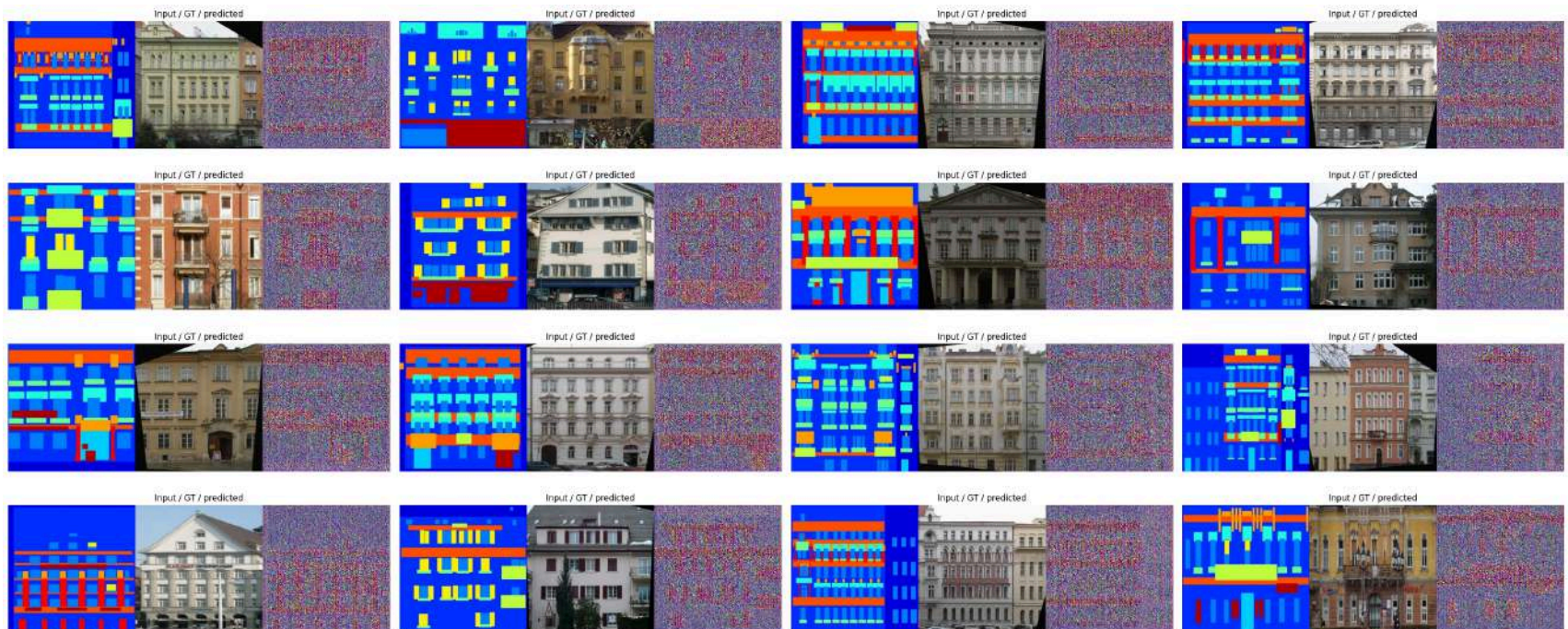
```

gs1 = gridspec.GridSpec(4, 4)
gs1.update(wspace=0.025)

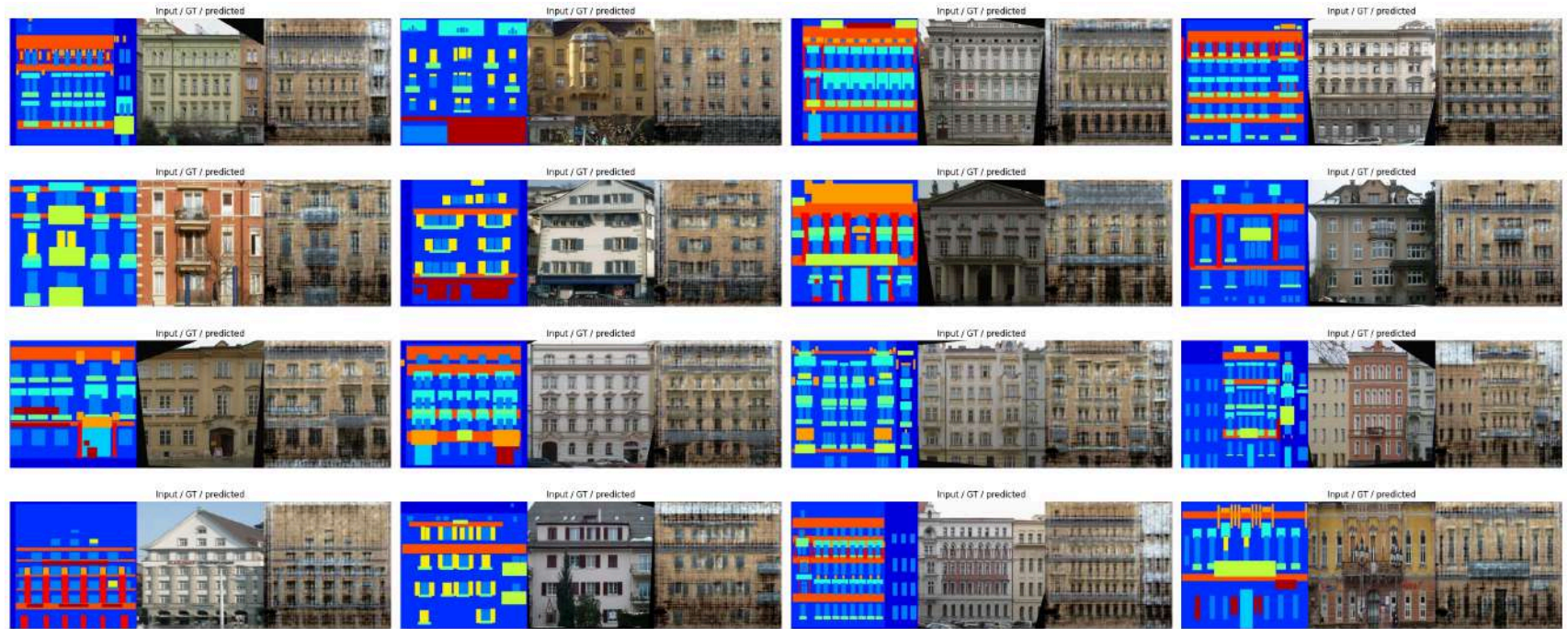
sampleX_vis = 0.5 * valX[:16][:, ::-1, :, :] + 0.5
sampleY = 0.5 * valY[:16][:, ::-1, :, :] + 0.5
sampleX = torch.tensor(valX[:16]).cuda()
pred_val = 0.5 * generator(sampleX).cpu().detach().numpy()[ :, ::-1, :, :] + 0.5
vis = np.concatenate([sampleX_vis, sampleY, pred_val], axis = 3)
for i in range(vis.shape[0]):
    ax1 = plt.subplot(gs1[i])
    plt.title('Input / GT / predicted')
    plt.axis('off')
    plt.imshow( vis[i].transpose(1, 2, 0) )
plt.show()

```

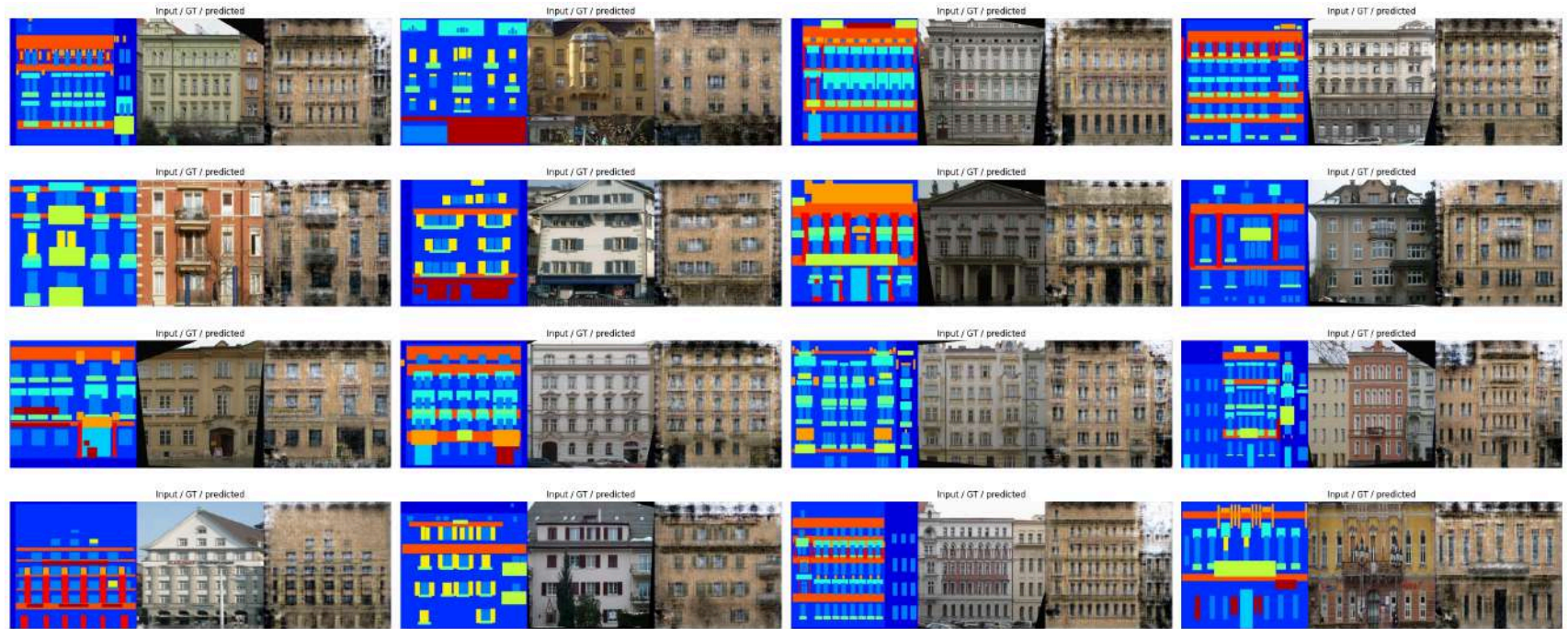
0% | 0/20001 [00:00<?, ?it/s]
0.7046584486961365 68.81037139892578



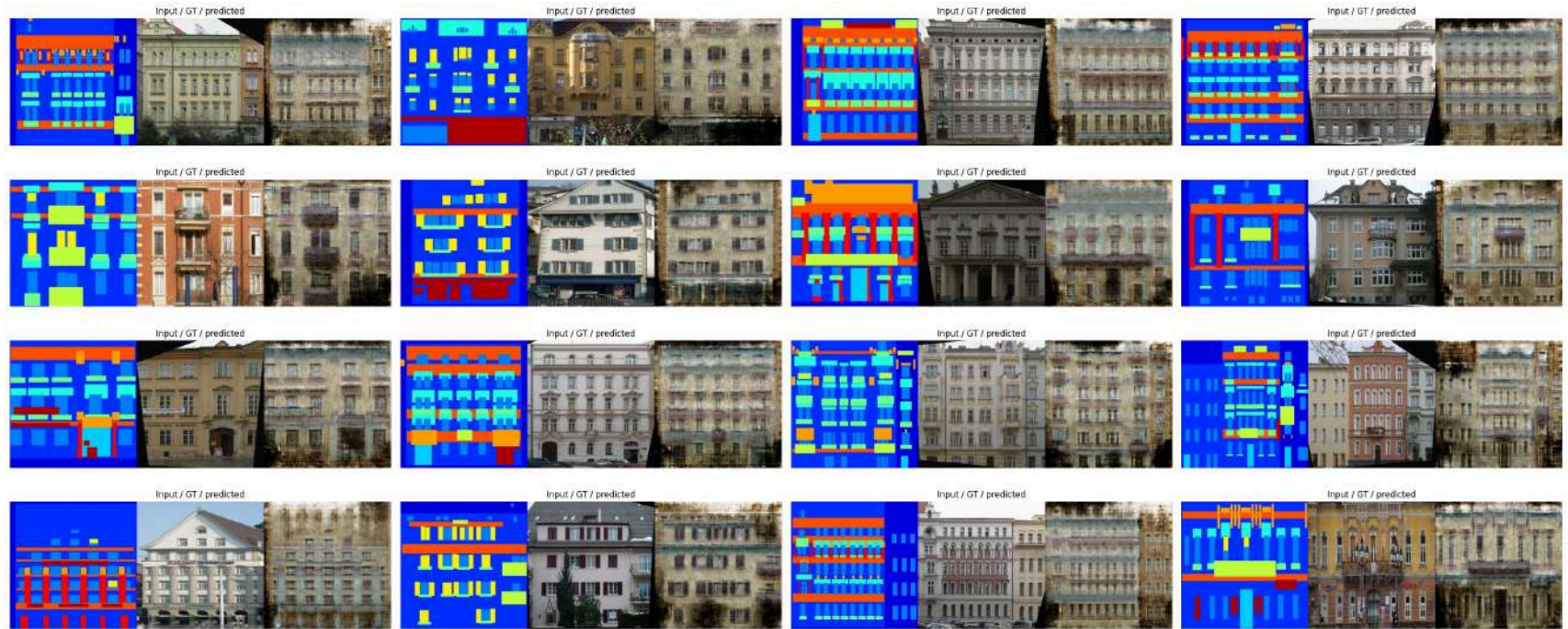
12% | 2497/20001 [01:17<08:34, 34.04it/s]
0.12420568615198135 36.6575813293457



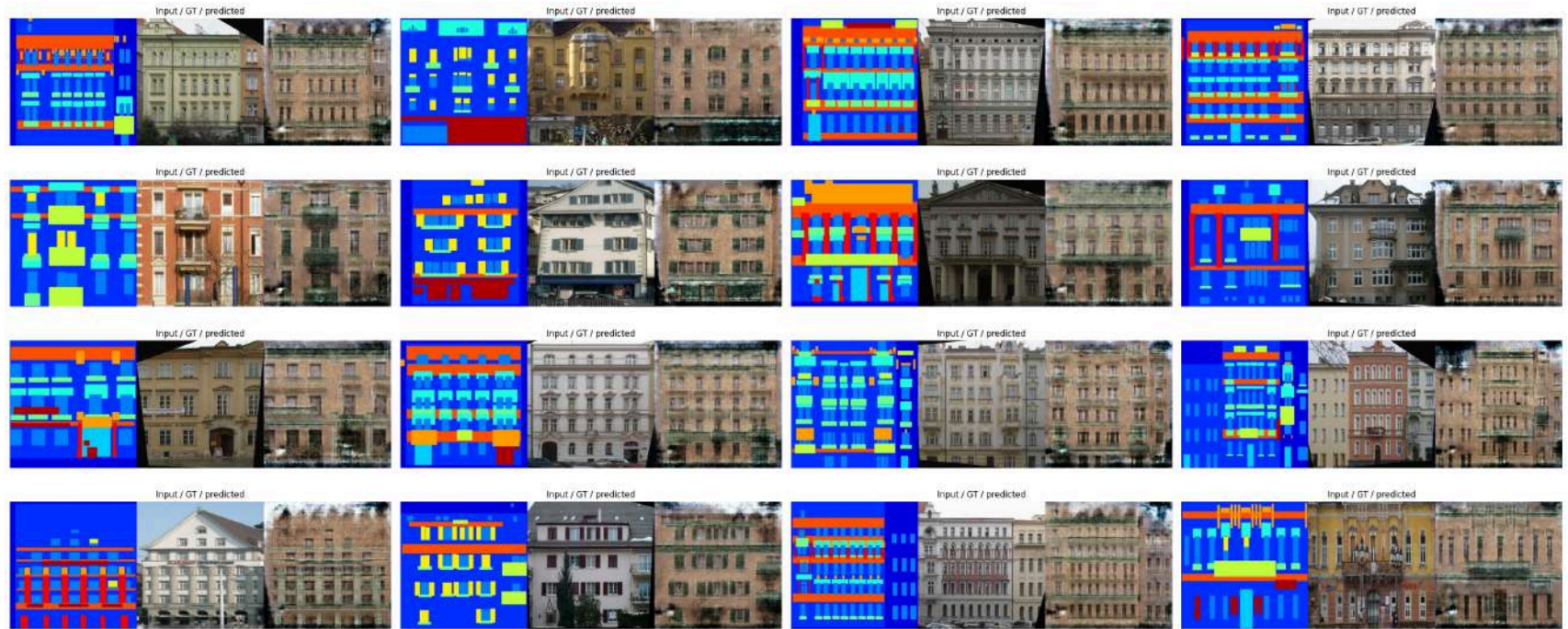
25% |  | 4997/20001 [02:33<07:46, 32.16it/s]
0.46604806184768677 29.617206573486328



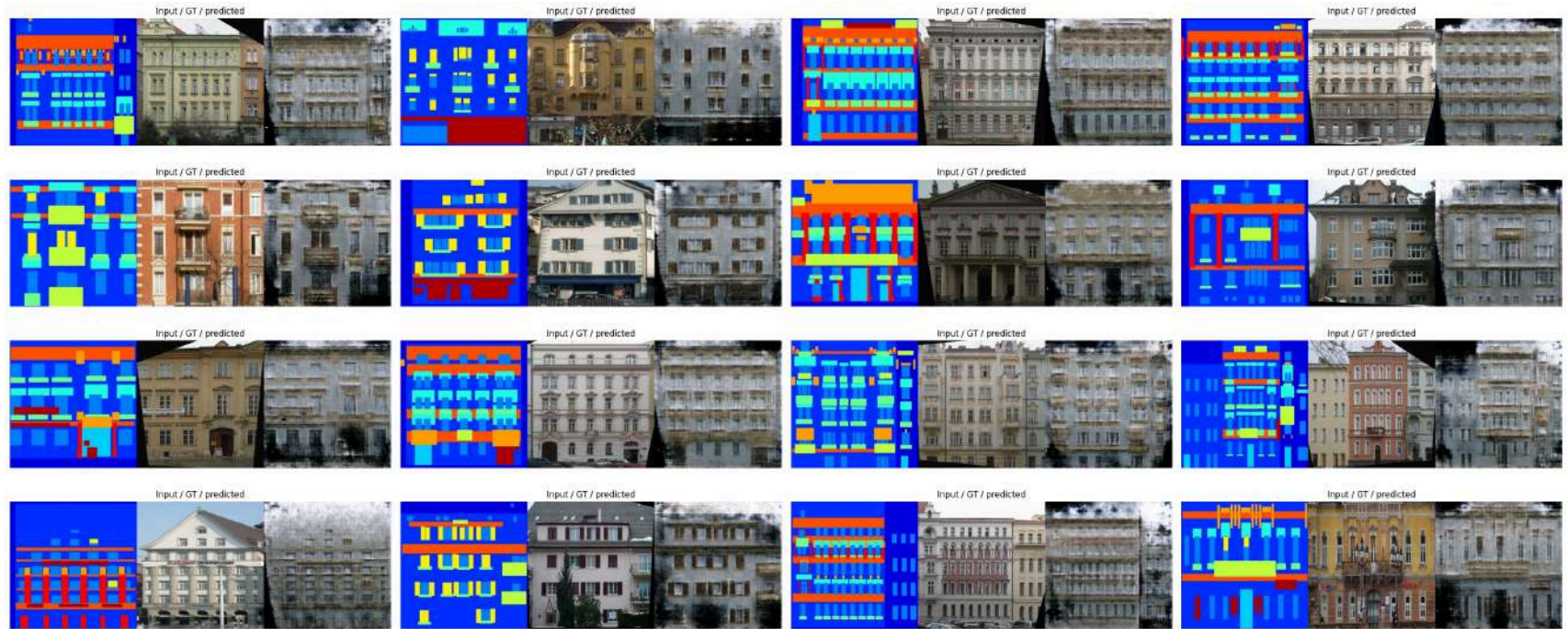
37% |  | 7498/20001 [03:49<06:02, 34.50it/s]
0.09933241456747055 30.93971824645996



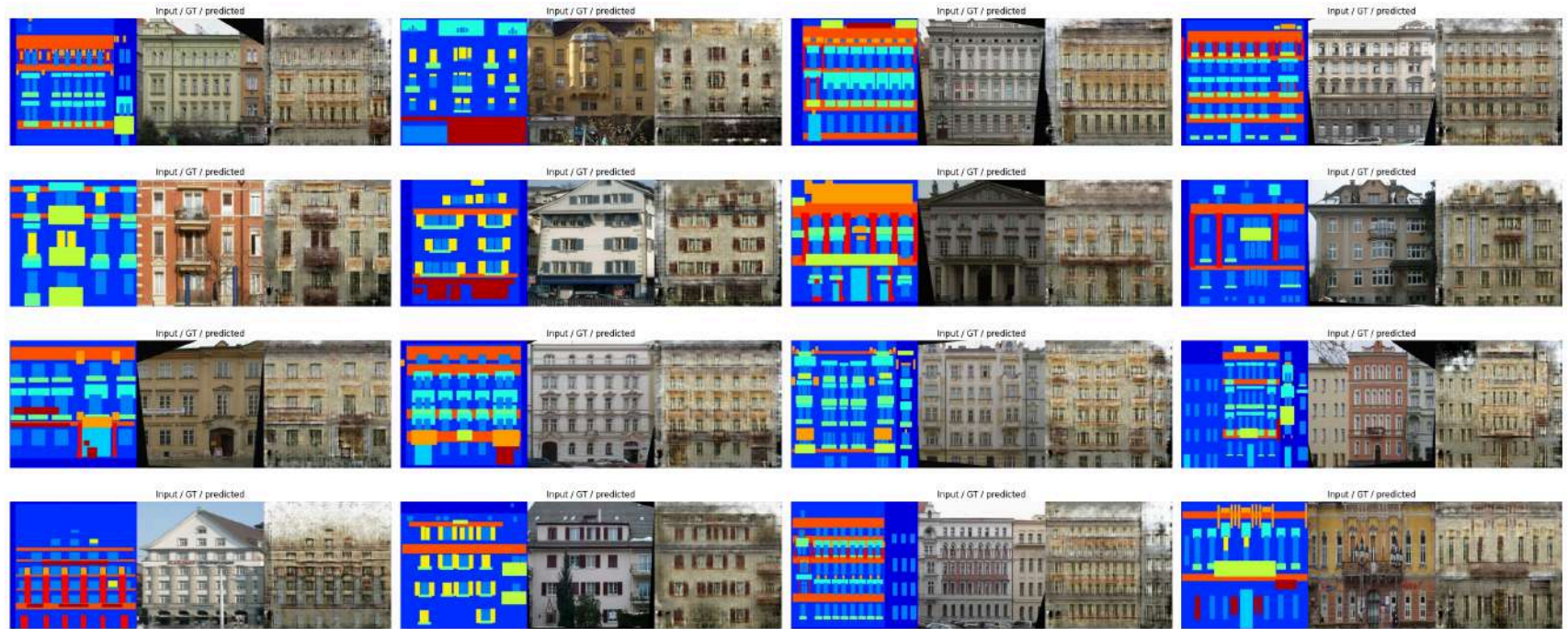
50% |  | 10000/20001 [05:10<04:56, 33.70it/s]
0.0021923023741692305 43.54263687133789



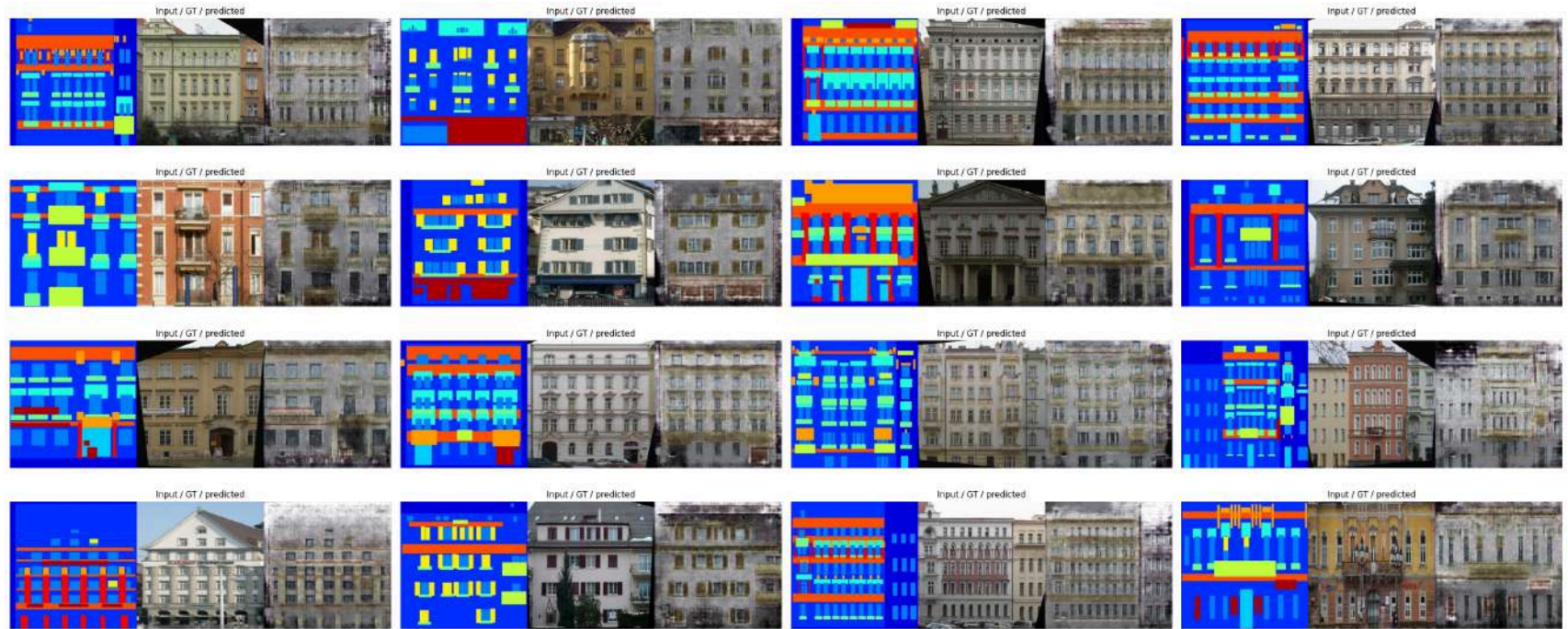
62% | ██████████ | 12500/20001 [06:30<06:32, 19.11it/s]
0.013867415487766266 38.445526123046875



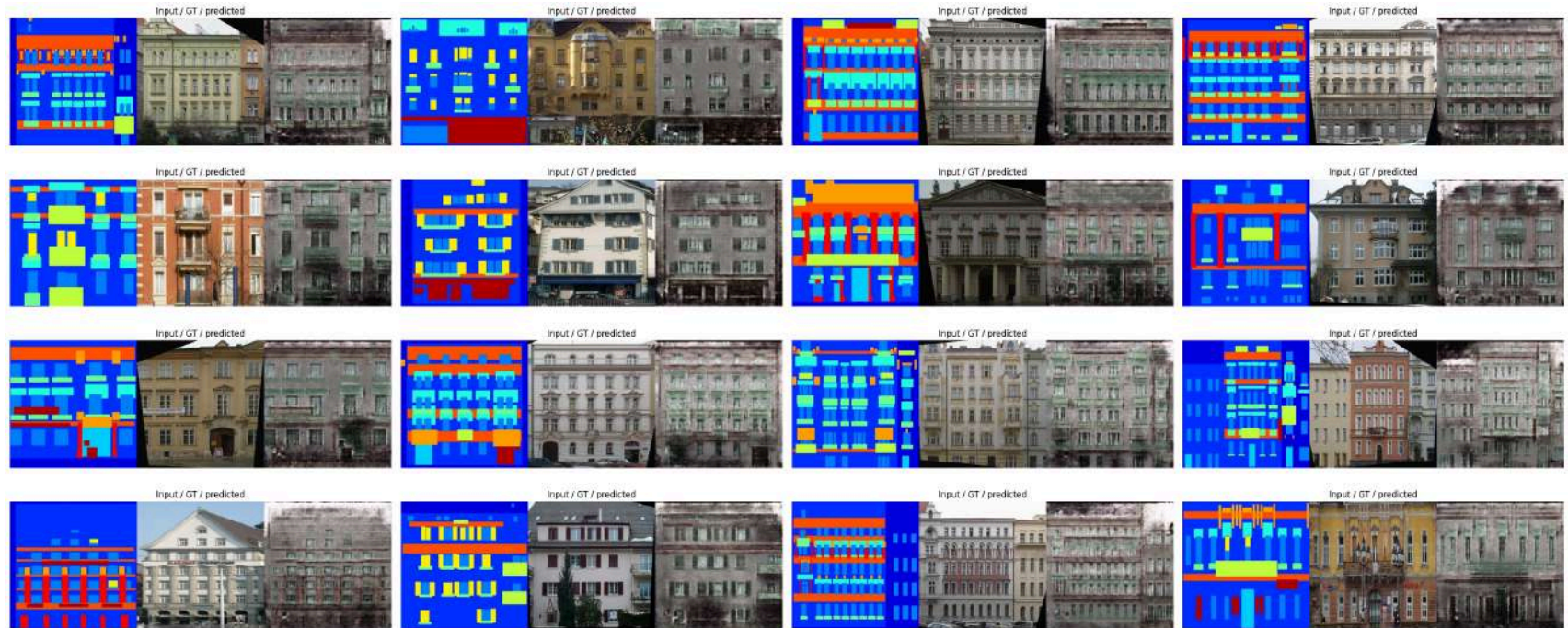
75% | ██████████ | 14998/20001 [07:34<02:37, 31.69it/s]
0.0038173161447048187 31.451173782348633



87% | ██████████ | 17500/20001 [08:45<01:07, 37.25it/s]
0.14196300506591797 34.240943908691406



100% | ██████████ | 19997/20001 [09:54<00:00, 39.43it/s]
0.08041093498468399 30.18451499938965



100%|██████████| 20001/20001 [09:56<00:00, 33.54it/s]
 100%|██████████| 20001/20001 [09:56<00:00, 33.54it/s]

(Optional)

Combine the WGAN-GP loss with the pix2pix objective.

In [17]: **pass**